

A. Overwatch - Telemetry Data Adapter - White-Box View

Overwatch

18 August, 2026

Table of Contents

Main Module	4
MQTT-Ingestion	4
Backpressure bei Persistierungsfehlern	5
Eskalation bei anhaltenden Persistierungsfehlern	5
Wiederherstellung nach Persistierungsfehlern.....	7
Buffer Manager	8
Process-Worker	11
Error handling.....	12
processErrorCase	12
Persistenter Nachrichtenspeicher	14
Entitäten	16
OriginalMessage	16
CanonicalMessage	18
DeliveryRecord	19
ReceiverState	21
ErrorCase.....	22
SourceProcessingState.....	24
ErrorHandling Package	27
Inhalt des error.log.....	27
Diagnosepaket	28
Schreibablauf	29
Fehlerklassifikation	29
Persistierung der MQTT-Message schlägt fehl:	29
Drohne kann noch nicht identifiziert werden:	29
Drohne bleibt beim Herunterfahren oder nach einem unsicheren Neustart nicht identifiziert:.....	29
Validierung oder Transformation schlägt fehl:	29
Empfänger ist nicht verfügbar:	30
Adapter stürzt ab:	30
Persistenter Speicher ist voll oder nicht verfügbar:	30

Empfänger bestätigt Speicherung, aber HTTP-Antwort geht verloren:.....	30
Identifier-Package.....	31
Validator-Package	34
Validierung der Telemetrierohdaten	34
Validierung der transformierten Telemetriedaten	35
Transformer-Package.....	35
DeliveryService-Package	36
Normative Zustandsübergänge	38
Wesentliche Funktionen	41
processNextDelivery.....	41
buildHttpMessage	42
sendHttpMessage	44
handleDeliveryResult	44
checkReceiverHealth.....	46
Idempotenz	47
Database-Package.....	48

Main Module

Das Hauptmodul initialisiert und koordiniert den Telemetry Data Adapter. Seine Aufgaben im Detail sind:

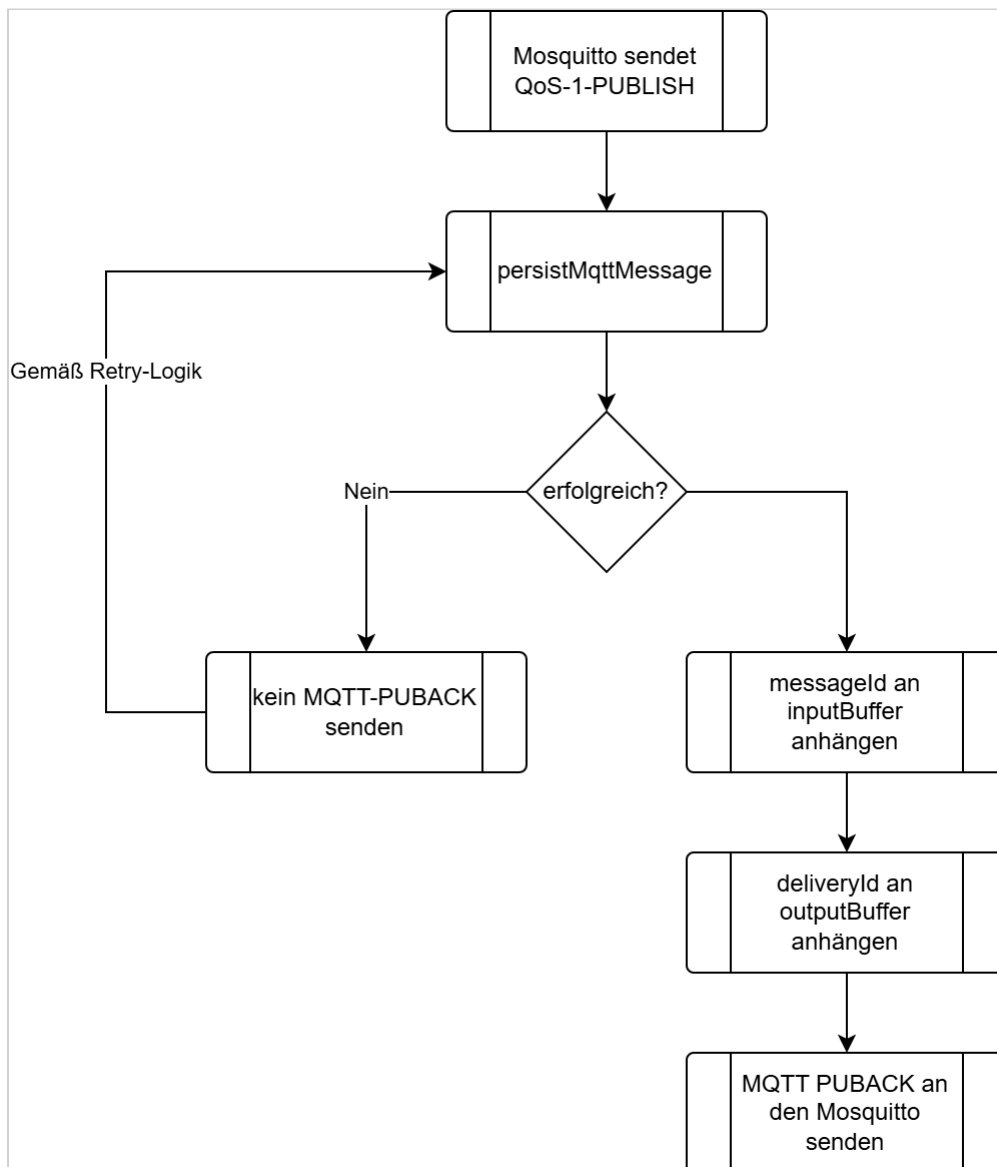
- stellt die MQTT-5-Verbindung zu Mosquitto her und verwendet eine persistente MQTT-Session,
- abonniert die konfigurierten Telemetrie-Topics mit QoS 1, um MQTT-Nutzlasten zu empfangen,
- startet und überwacht den `Process-Worker`,
- startet und überwacht den `DeliveryService`,
- koordiniert das geordnete Herunterfahren,
- koordiniert die Wiederherstellung beim Startup

MQTT-Ingestion

Für den ersten Persistierungsversuch und alle Wiederholungsversuche wird dieselbe logische Operation `persistMqttMessage` verwendet. `persistMqttMessage` führt genau eine Datenbanktransaktion aus. Innerhalb dieser Transaktion:

- wird die `OriginalMessage` mit `processingStatus = pending` und der nächsten `ingressSequence` erzeugt.
- wird der zugehörige `DeliveryRecord` für die Originaltelemetrie mit `status = pending` und der nächsten globalen `deliverySequence` erzeugt.

Die Operation ist nur erfolgreich, wenn beide Datensätze gemeinsam committed wurden. Bei einem Fehler wird die gesamte Transaktion zurückgerollt. Nach einem erfolgreichen Commit werden `messageId` und `deliveryId` gemäß den Regeln des Buffer Managers in `inputBuffer` beziehungsweise `outputBuffer` aufgenommen. Erst danach darf MQTT-PUBACK gesendet werden.



Backpressure bei Persistierungsfehlern

Kann die MQTT-Message nicht persistent gespeichert werden, sendet der Telemetry Data Adapter kein MQTT-PUBACK. Da `Receive Maximum = 1` verwendet wird, darf Mosquitto währenddessen keine weitere noch nicht bestätigte QoS-1- oder QoS-2-Message an den Telemetry Data Adapter senden. Das Zurückhalten von PUBACK zusammen mit `Receive Maximum = 1` bildet die Backpressure-Strategie.

Eskalation bei anhaltenden Persistierungsfehlern

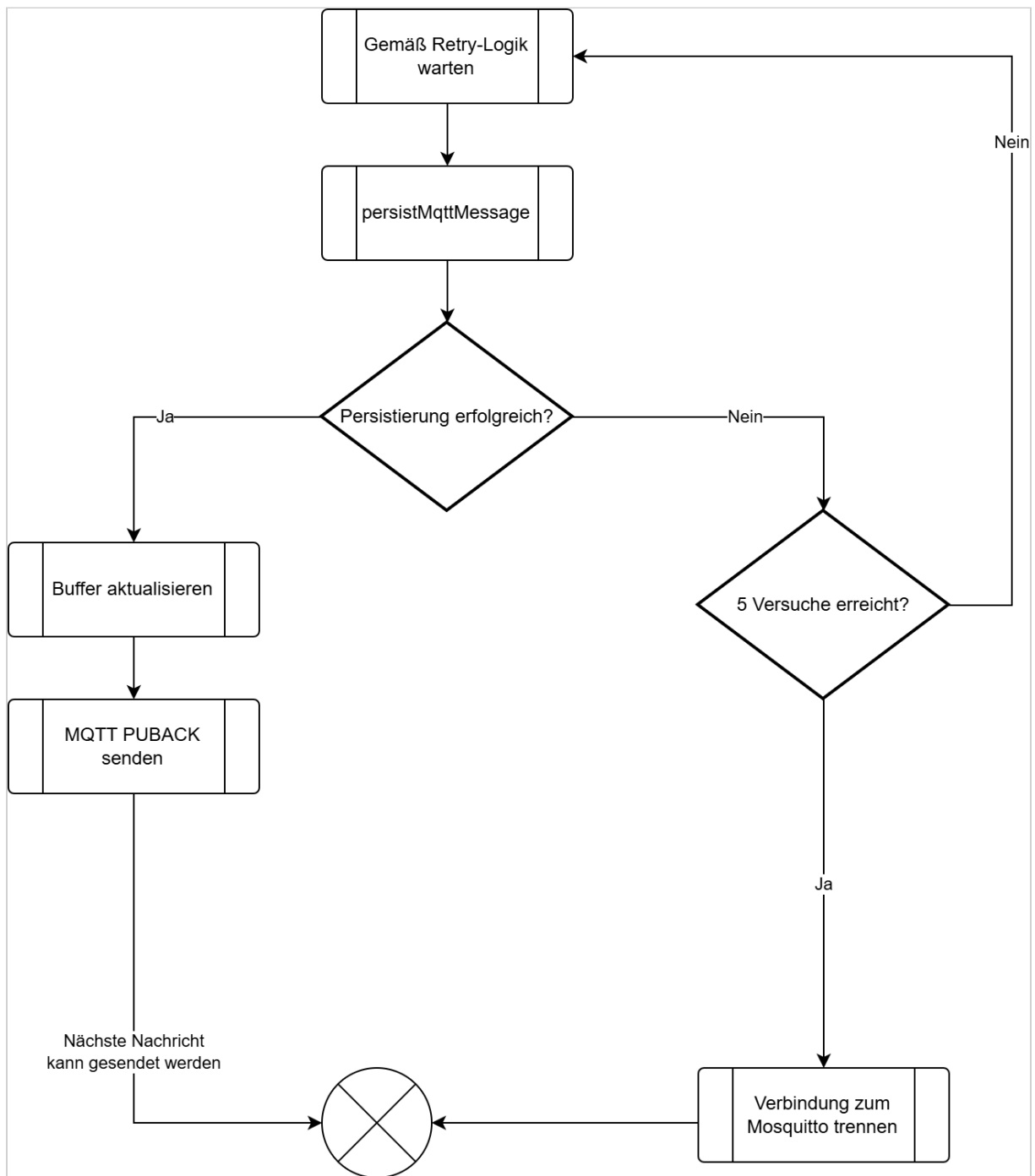
Kann die Message trotz der konfigurierten Anzahl von max. 5 Persistierungsversuchen nicht gespeichert werden, trennt der Adapter die MQTT-Verbindung, ohne die persistente Session oder das

Abonnement zu löschen. Die Trennung der Verbindung ist keine Backpressure-Maßnahme, sondern die Eskalation eines anhaltenden Problems.

Der erste Persistierungsversuch wird unmittelbar ausgeführt. Schlägt er fehl, führt der Adapter höchstens 4 weitere Versuche aus:

- Wiederherstellungsversuch 1 nach 100 Millisekunden,
- Wiederherstellungsversuch 2 nach 500 Millisekunden,
- Wiederherstellungsversuch 3 nach 1.000 Millisekunden,
- Wiederherstellungsversuch 4 nach 2.000 Millisekunden

Damit werden höchstens 5 Versuche ausgeführt.



Wiederherstellung nach Persistierungsfehlern

Nachdem das Problem administrativ behoben wurde, kann sich der Telemetry Data Adapter mit derselben `clientId`, `Clean Start = false` und dem bestehenden `Session Expiry Interval` erneut mit Mosquitto verbinden.

Buffer Manager

Der Buffer Manager koordiniert drei In-Memory-FIFO-Buffer. Dies sind:

- `inputBuffer` : globale FIFO-Warteschlange für noch zu verarbeitende `OriginalMessage`-Datensätze
- `clipboardBuffer` : nach `sourceId` partitionierte FIFO-Warteschlange für `OriginalMessage`-Datensätze, die auf eine eindeutige Identifikation ihrer Quelle warten
- `outputBuffer` : globale FIFO-Warteschlange aller noch nicht bestätigten `DeliveryRecord`-Datensätze

Die Buffer enthalten ausschließlich Identifier. Der persistente Nachrichtenspeicher bleibt die maßgebliche Zustandsquelle des Adapters.

Für jeden Buffer gelten außerhalb eines gerade ausgeführten Zustandsübergangs folgende Invarianten:

- Eine `messageId` befindet sich genau dann im `inputBuffer`, wenn die zugehörige `OriginalMessage.processingStatus = pending` besitzt und aktuell nicht vom `Process-Worker` verarbeitet wird.
- Eine `messageId` befindet sich genau dann in einer Partition des `clipboardBuffer`, wenn die zugehörige `OriginalMessage.processingStatus = waiting-for-identification` besitzt.
- Eine `deliveryId` befindet sich genau dann im `outputBuffer`, wenn der zugehörige `DeliveryRecord.status != acknowledged` ist.

Die Aufgaben des Buffer Manager umfassen:

- stellt FIFO-Operationen zum Einfügen und Lesen des Kopfes (peek) und Entfernen bereitzustellen,
- signalisiert Konsumenten, wenn Arbeit verfügbar ist,
- partitioniert den Clipboard-Buffer nach `sourceId`,
- verhindert, dass derselbe persistente Datensatz mehrfach eingeplant wird,
- baut `inputBuffer` und `outputBuffer` nach dem Start aus persistenten Datensätzen wieder auf und initialisiert den `clipboardBuffer` leer,
- verhindert widersprüchliche Buffer-Changes während Übergängen beim Herunterfahren.

Buffer	Partitionierung	Identifizier	Bedeutung
inputBuffer		message Id	Enthält alle zur Verarbeitung anstehenden OriginalMessage-Datensätze mit <code>processingStatus = pending</code> . Die Reihenfolge richtet sich nach <code>OriginalMessage.ingressSequence</code> aufsteigend. Der Process-Worker verarbeitet immer den Datensatz am Kopf des Buffers.
clipboardBuffer	sourceId	message Id	Enthält OriginalMessage-Datensätze mit <code>processingStatus = waiting-for-identification</code> . Innerhalb jeder <code>sourceId</code> -Partition erfolgt die Reihenfolge nach <code>ingressSequence</code> aufsteigend. Der Buffer ist kein persistenter Zustand und wird beim Startup nicht rekonstruiert.
outputBuffer		deliveryId	Enthält jeden <code>DeliveryRecord</code> mit <code>status != acknowledged</code> genau einmal. Die Reihenfolge richtet sich global nach <code>DeliveryRecord.deliverySequence</code> aufsteigend. Der Datensatz am Kopf des Buffers ist damit immer der führende unbestätigte <code>DeliveryRecord</code> . Ein Eintrag verbleibt im <code>outputBuffer</code> , solange sein Status <code>pending</code> , <code>in-flight</code> , <code>retry-wait</code> oder <code>blocked</code> ist. Er wird erst entfernt, nachdem der zugehörige <code>DeliveryRecord</code> persistent den Status <code>acknowledged</code> erhalten hat.

Ist für eine `sourceId` keine vertrauenswürdige `deviceId` bekannt (`SourceProcessingState.identificationStatus = unidentified`), versucht der Process-Worker zunächst, die aktuell verarbeitete Nachricht zur Identifikation der Drohne zu verwenden. Kann die Drohne anhand dieser Nachricht nicht eindeutig identifiziert werden, setzt der Process-Worker `OriginalMessage.processingStatus = waiting-for-identification` und fügt die `messageId` am Ende der zugehörigen `sourceId`-Partition des `clipboardBuffers` ein.

Kann eine spätere Nachricht derselben `sourceId` die Drohne eindeutig identifizieren, aktualisiert der Process-Worker zunächst den `SourceProcessingState`. Anschließend verarbeitet er die bereits wartenden Nachrichten dieser `clipboardBuffer`-Partition in Reihenfolge ihrer `ingressSequence`. Erst nachdem die Partition vollständig abgearbeitet wurde, setzt er die Verarbeitung der Nachricht fort, durch die die Identifikation möglich wurde.

Bevor der Process-Worker eine `OriginalMessage` aus einer `clipboardBuffer`-Partition weiterverarbeitet, setzt er `OriginalMessage.processingStatus` von `waiting-for-`

identification auf processing. Nach erfolgreicher Persistierung entfernt er die messageId aus der betreffenden clipboardBuffer-Partition. Anschließend durchläuft die Nachricht die Rohdatenvalidierung, Transformation und Validierung der transformierten Daten. Nach Abschluss der Verarbeitung setzt der Process-Worker den entsprechenden terminalen processingStatus und fährt mit dem nächsten Eintrag der Partition fort.

Beispiel:

```
Nachricht 0 → Drohne kann nicht identifiziert werden  
Nachricht 1 → Drohne kann nicht identifiziert werden  
Nachricht 2 → Drohne kann nicht identifiziert werden  
Nachricht 3 → identifiziert die Drohne
```

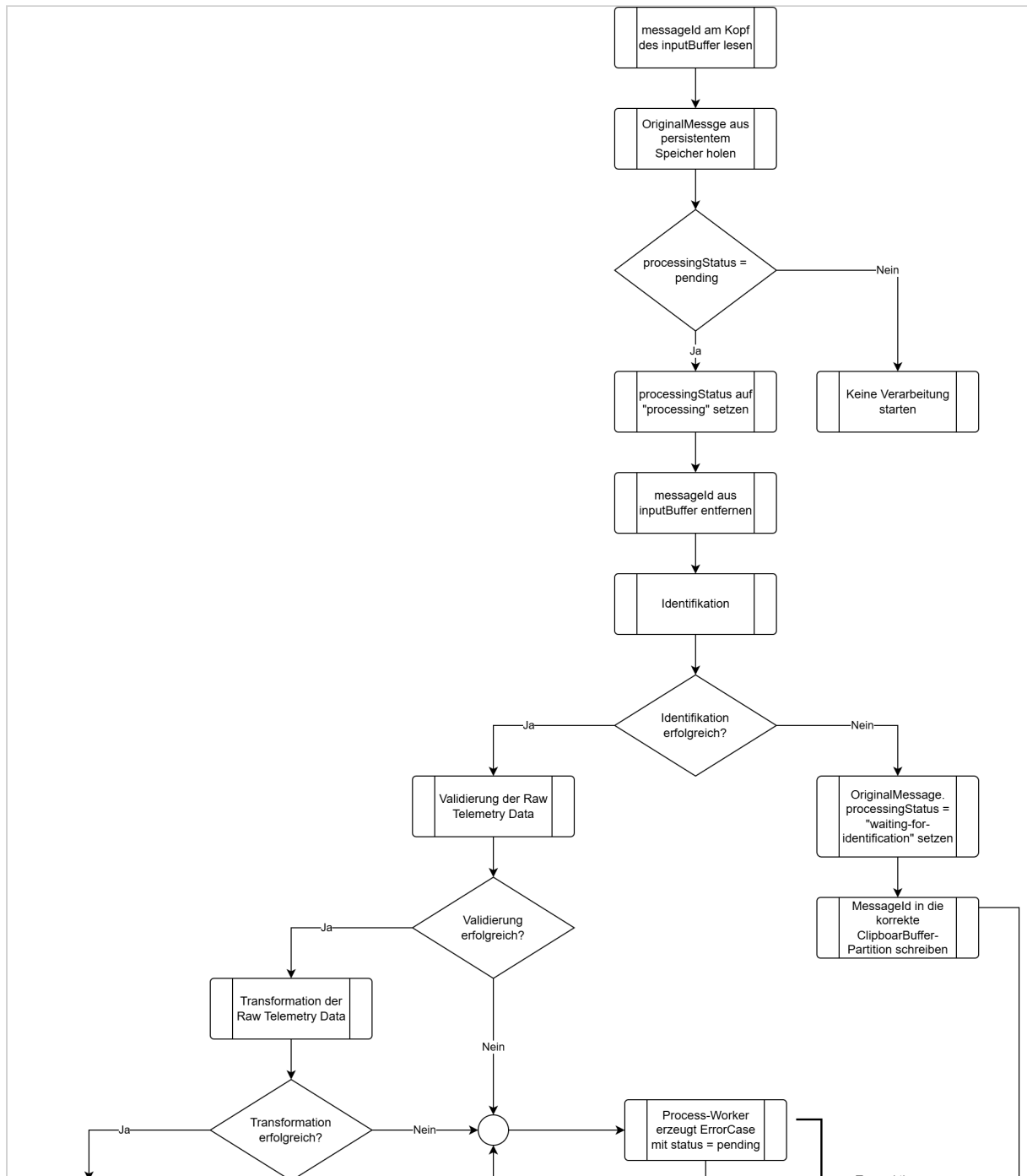
Nachdem Nachricht 3 die Drohne identifiziert hat, aktualisiert der Process-Worker den SourceProcessingState. Anschließend verarbeitet er die Nachrichten 0, 1 und 2 aus der betreffenden clipboardBuffer-Partition in aufsteigender Reihenfolge ihrer ingressSequence. Erst danach setzt er die Verarbeitung von Nachricht 3 fort. Die Nachrichten 0, 1 und 2 werden dabei nicht erneut in den inputBuffer eingestellt.

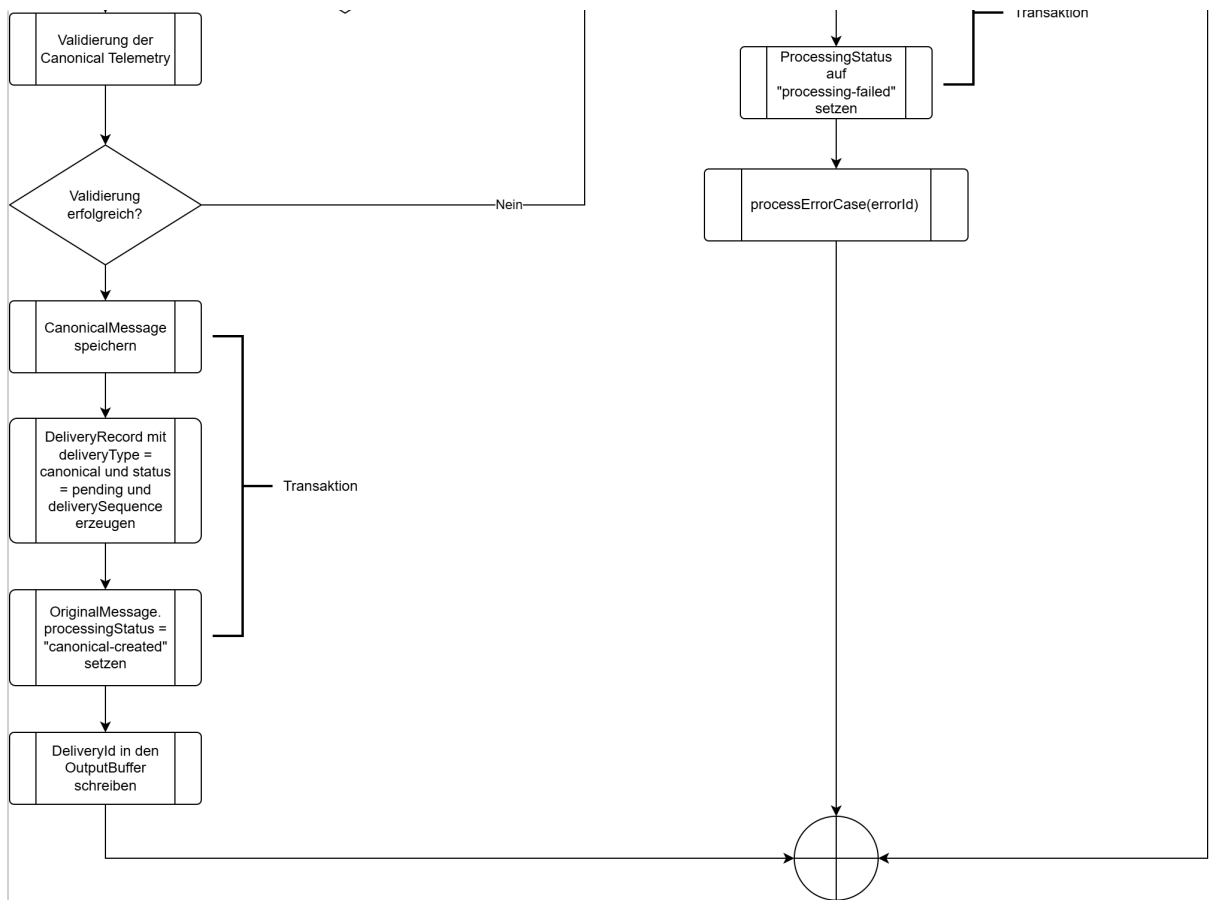
Die DeliveryRecords der Telemetrierothdaten sind davon unabhängig und können vom Empfänger bereits bestätigt worden sein.

Process-Worker

Der Process-Worker führt den Verarbeitungspfad für transformierte Telemetrie aus und bleibt während der Laufzeit des Adapters aktiv. Seine Tätigkeiten umfassen die Identifikation des Senders (Drohne), die Validierung der Telemetrierohdaten, die Transformation der Rohdaten in das interne Datenformat und die Validierung der transformierten Telemetrie. Hierfür greift er auf die Packages Identifier, Validator und Transformer zu.

Der Prozess führt außerdem die Fehlerbehandlung für fehlgeschlagene Verarbeitungsvorgänge durch.





Die persistente Erzeugung des `DeliveryRecord` für die Telemetrierohdaten ist von der Transformation unabhängig. Deshalb werden die Rohdaten schon vor dem Verarbeitungsprozess dauerhaft zur Zustellung vorgemerkt. Die Originalnachricht wird deshalb auch dann zugestellt, wenn:

- die Drohne nicht identifiziert werden kann,
- die Rohdatenvalidierung fehlschlägt,
- die Transformation fehlschlägt,
- die Validierung der transformierten Daten fehlschlägt.

Error handling

processErrorCase

Der `Process-Worker` behandelt einen `ErrorCase` über die Operation `processErrorCase(errorId)`. Dieselbe Operation wird sowohl für den ersten Diagnoseversuch als auch für alle späteren Wiederholungsversuche verwendet.

Zu Beginn eines Diagnoseversuchs aktualisiert der `Process-Worker` den `ErrorCase` persistent:

- `ErrorCase.status = processing,`

- `ErrorCase.attemptCount` um 1 erhöhen,
- `ErrorCase.lastAttemptAt` auf den aktuellen Zeitpunkt setzen,
- `ErrorCase.nextAttemptAt = null` setzen.

Anschließend:

- erzeugt er das Diagnosepaket unter Verwendung derselben `errorId` zunächst an einem temporären Speicherort,
- prüft er das Diagnosepaket auf Vollständigkeit und Prüfsummen,
- verschiebt bzw. ersetzt er das Diagnosepaket nach erfolgreicher Prüfung an seinen endgültigen Speicherort,
- speichert er `ErrorCase.diagnosticPackageLocation`,
- schreibt er einen strukturierten Eintrag in die Datei `error.log`,
- setzt er `ErrorCase.status` auf `completed` und `ErrorCase.completedAt` = aktueller Zeitpunkt.

Schlägt eine erforderliche Diagnoseoperation fehl, setzt der `Process-Worker`

`ErrorCase.status = retry-wait` und `ErrorCase.nextAttemptAt = aktueller Zeitpunkt + 5 Sekunden`. Der aktuelle Diagnoseversuch ist damit beendet. Der `ErrorCase-Retry-Timer` führt zum vorgesehenen Zeitpunkt einen neuen Aufruf von `processErrorCase(errorId)` aus.

Sollte der `Telemetry Data Adapter` zwischenzeitlich beendet worden sein (abgestürzt sein), so bleibt `ErrorCase.nextAttemptAt` persistent erhalten. Ist der Zeitpunkt beim Startup bereits erreicht, kann der Diagnoseversuch unmittelbar wieder aufgenommen werden. Liegt `ErrorCase.nextAttemptAt` noch in der Zukunft, verbleibt der `ErrorCase` bis zu diesem Zeitpunkt im Zustand `retry-wait`. Diagnoseversuche besitzen keine fest maximale Versuchszahl. Sie werden wiederholt, bis `processErrorCase` erfolgreich abgeschlossen wurde oder administrativ eingegriffen wird.

Fehlgeschlagene Diagnoseoperationen werden, soweit `error.log` schreibbar ist, dort protokolliert. Ein Fehler beim Schreiben von `error.log` selbst wird ausschließlich über den persistenten `ErrorCase` und dessen Wiederholungszustand erfasst. Hierfür wird kein rekursiver Logeintrag erzwungen.

Persistenter Nachrichtenspeicher

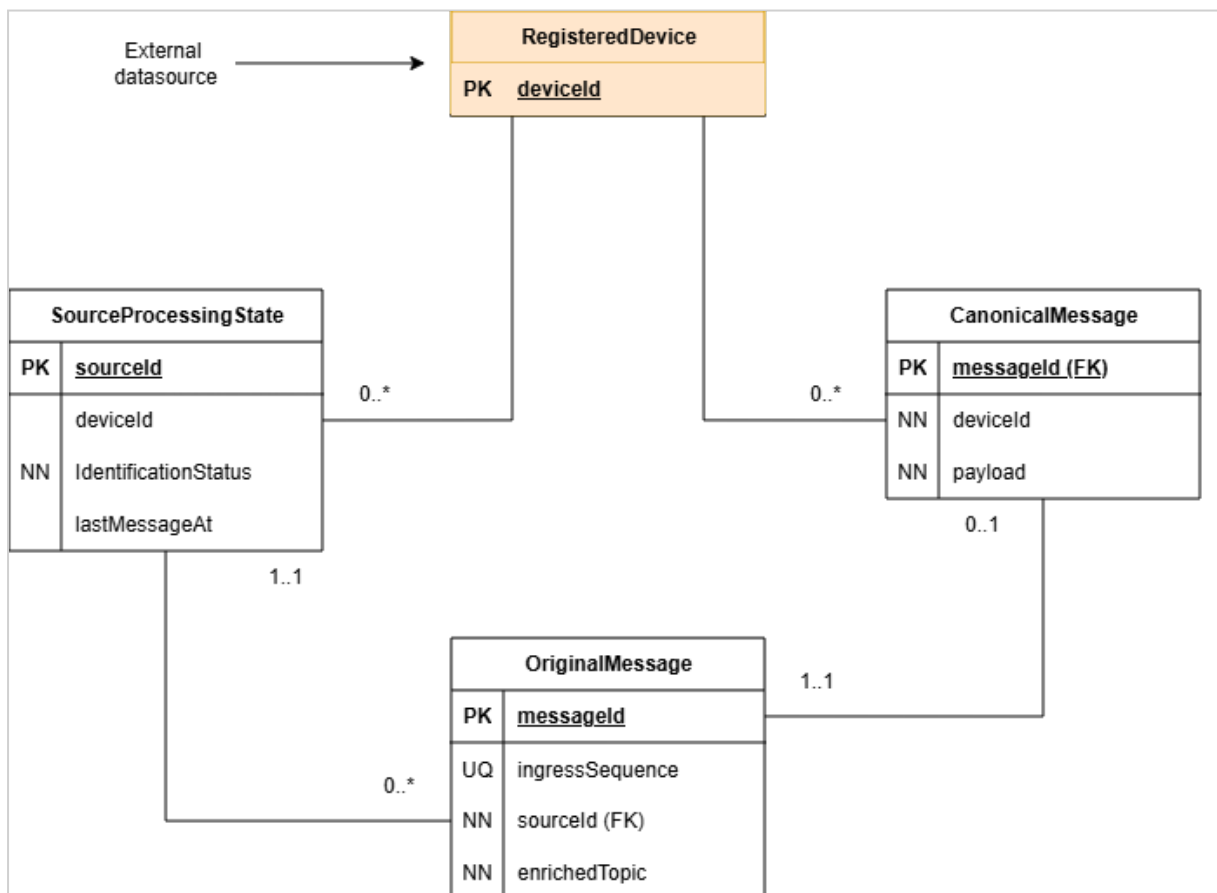
Der persistente Nachrichtenspeicher ist die maßgebliche Zustandsquelle des Adapters (Single Source of Truth). Zu seinen Aufgaben zählen:

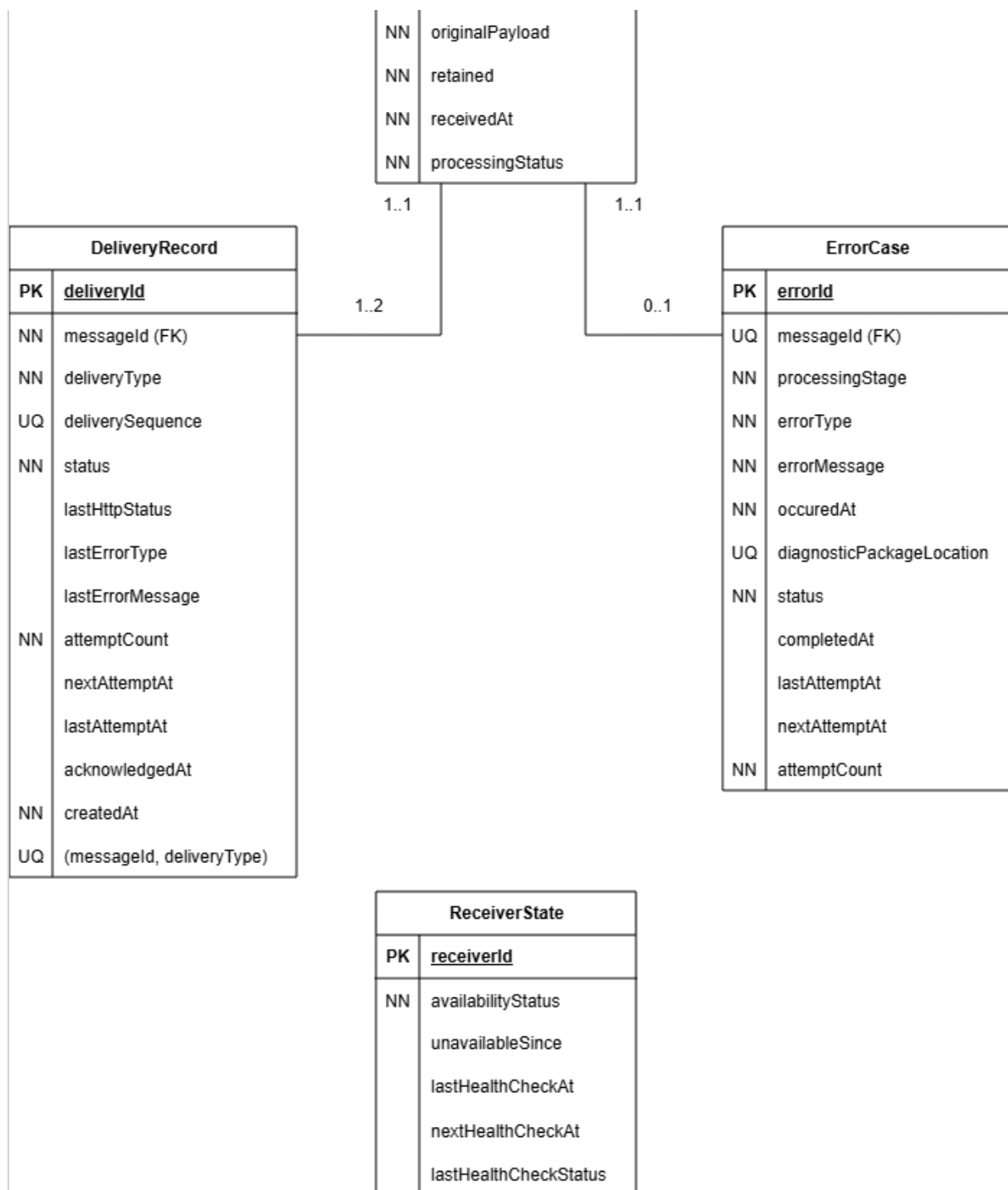
- speichern akzeptierter Telemetrierohdaten,
- erzeugen eines DeliveryRecords für Telemetrierohdaten in derselben Transaktion,
- speichern erfolgreich transformierter und validierter Telemetrie,
- speichern des Status der Verarbeitung von Telemetriedaten (`OriginalMessage.processingStatus`),
- speichern der HTTP-Zustellversuche, Wiederholungsplanung und Bestätigungen,
- speichern von Verarbeitungsfehlern,
- unterstützen der Wiederherstellung nach einem Prozess- oder Hostneustart,
- verhindern der Löschung akzeptierter Daten vor der Bestätigung durch den Empfänger.

Datenbank und zugrunde liegendes Dateisystem müssen Synchronisierungs- und Flush-Operationen korrekt ausführen. Eine erfolgreiche Datenbankbestätigung ist die lokale Persistenzgrenze.

Datenmodell

Alle maßgeblichen Datensätze werden im persistenten Nachrichtenspeicher abgelegt. Die FIFO-Buffer enthalten ausschließlich Identifier, die auf diese Datensätze verweisen.





Zusätzliche Semantik, die nicht im Diagramm abbildbar ist:

- Ein **DeliveryRecord** mit **DeliveryRecord.deliveryType = canonical** kann nur existieren, wenn ein Datensatz in **CanonicalMessage** mit der gleichen **messageId** existiert.

- Die Datensätze in `CanonicalMessage` und `DeliveryRecord` müssen in der gleichen Transaktion generiert werden, in der auch `OriginalMessage.processingStatus = canonical-created` gesetzt wird.
- Der `DeliveryRecord` mit `deliveryType = original` muss in der gleichen Transaktion generiert werden wie der Datensatz in `OriginalMessage`.
- `ErrorCase.diagnosticPackageLocation` kann nur dann `null` sein, wenn `ErrorCase.status` einen der Werte `pending`, `processing` oder `retry-wait` hat.
- Hat `ErrorCase.status` den Wert `completed`, muss `ErrorCase.diagnosticPackageLocation` gesetzt sein.
- `DeliveryRecord.acknowledgedAt` muss gesetzt sein, wenn `DeliveryRecord.status = acknowledged`.
- `DeliveryRecord.nextAttemptAt` muss gesetzt sein, wenn `DeliveryRecord.status = retry-wait`.
- `ReceiverState.nextHealthCheckAt` muss gesetzt sein, wenn `ReceiverState.availabilityStatus = unavailable`.
- `ErrorCase.nextAttemptAt` muss gesetzt sein, wenn `ErrorCase.status = retry-wait`.
- `ErrorCase.completedAt` muss gesetzt sein, wenn `ErrorCase.status = completed`.

Entitäten

OriginalMessage

Sie speichert die MQTT-Nachricht, die der Adapter akzeptiert hat.

Attribut	Erklärung
<code>messageId</code>	Primärschlüssel der akzeptierten Nachricht. Er verknüpft Originaltelemetrie, transformierte Telemetrie, Verarbeitungsfehler und Zustelldatensätze.
<code>ingressSequence</code>	Fortlaufende Nummer, die bei der Annahme der Telemetrierohdaten vergeben wird. Sie bewahrt die Reihenfolge der Originalnachrichten, ohne sich auf Zeitstempel zu verlassen.
<code>sourceId</code>	Identifiziert den Controller beziehungsweise die veröffentlichende Quelle. Der Wert wird aus dem Namensraum extrahiert, den das Mosquitto Topic Routing Plugin ergänzt.

Attribut	Erklärung
enrichedTopic	Speichert den vollständig angereicherten MQTT-Topic. Der ursprüngliche Topic muss daraus verlustfrei rekonstruiert werden können.
originalPayload	Speichert die unveränderte MQTT-Payload (Originaltelemetriedaten / Telemetrirohdaten).
retained	Bewahrt das Retained-Flag der eingehenden MQTT-PUBLISH-Message auf.
receivedAt	Zeitpunkt, zu dem der Adapter die MQTT-Message empfangen hat.
processingStatus	Verfolgt Zustand und Endergebnis des Verarbeitungspaths für transformierte Telemetrie. Der Wert beeinflusst die Zustellung der Originalnachricht nicht.

Zulässige Werte für `processingStatus` :

Wert	Bedeutung
pending	Die <code>OriginalMessage</code> wurde persistent gespeichert, aber die Transformation wurde noch nicht begonnen. Solange die Nachricht nicht gerade vom <code>Process-Worker</code> übernommen wird, befindet sich ihre <code>messageId</code> genau einmal im <code>inputBuffer</code> .
waiting-for-identification	Die Nachricht kann noch nicht verarbeitet werden, weil die Quelle beziehungsweise die Drohne nicht eindeutig identifiziert werden konnte. Die <code>messageId</code> befindet sich genau einmal in der der <code>sourceId</code> entsprechenden Partition des <code>clipboardBuffer</code> und wartet auf eine spätere eindeutige Identifizierung.
processing	Die Nachricht wird aktuell identifiziert, validiert oder transformiert. Dieser Zustand ist meist nur vorübergehend. Die Nachricht befindet sich weder im <code>inputBuffer</code> noch im <code>clipboardBuffer</code> . Sie wird aktuell vom <code>Process-Worker</code> verarbeitet.
canonical-created	Identifikation, Rohdatenvalidierung, Transformation und Validierung der transformierten Telemetrie waren erfolgreich. Eine <code>CanonicalMessage</code> und der zugehörige <code>DeliveryRecord</code> wurden persistent erzeugt. Die <code>messageId</code> befindet sich in keinem Buffer.

Wert	Bedeutung
processing-failed	Die Verarbeitung der Telemetriedaten ist aufgrund eines Validierungs-, Transformations- oder sonstigen Verarbeitungsfehlers endgültig fehlgeschlagen. Der Process-Worker hat den zugehörigen <code>ErrorCase</code> persistent gespeichert. Der Bearbeitungszustand von <code>error.log</code> und Diagnosepaket wird separat über <code>ErrorCase.status</code> verfolgt. Die Originaltelemetrie bleibt davon unberührt. Die <code>messageId</code> befindet sich in keinem Buffer.
abandoned-at-shutdown	Die Nachricht konnte beim Herunterfahren nicht mehr eindeutig identifiziert und deshalb nicht verarbeitet werden. Der Verarbeitungspfad für transformierte Telemetrie wurde kontrolliert beendet und als Fehler dokumentiert. Die <code>messageId</code> befindet sich in keinem Buffer.
abandoned-after-restart	Die Nachricht stammt aus einem früheren, nicht mehr vertrauenswürdigen Quellkontext. Nach dem Neustart konnte die frühere Zuordnung zwischen <code>sourceId</code> und <code>deviceId</code> nicht sicher wiederverwendet werden, weshalb die Verarbeitung abgebrochen wurde. Die <code>messageId</code> befindet sich in keinem Buffer.

Ein zugehöriger `ErrorCase` dokumentiert den Abbruch bei:

- `OriginalMessage.processingStatus = processing-failed`,
- `OriginalMessage.processingStatus = abandoned-at-shutdown`,
- `OriginalMessage.processingStatus = abandoned-after-restart`.

CanonicalMessage

Sie speichert das validierte Ergebnis einer erfolgreichen Verarbeitung der Telemetrierohdaten.

Attribut	Erklärung
<code>messageId</code>	Primärschlüssel der <code>CanonicalMessage</code> . Er verweist auf die <code>OriginalMessage</code> , aus der die <code>CanonicalMessage</code> abgeleitet wurde.
<code>deviceId</code>	Identifiziert die registrierte Drohne, zu der die Telemetrie gehört.
<code>payload</code>	Speichert die transformierten und validierten Telemetriedaten.

Ein solcher Datensatz existiert nur, wenn alle Verarbeitungsschritte erfolgreich waren. Nachrichten einer `sourceId` werden in der Reihenfolge ihres Eingangs verarbeitet. Clipboard-Nachrichten

werden vor der späteren Nachricht verarbeitet, die die erforderlichen Identifikationsinformationen geliefert hat.

DeliveryRecord

Er speichert den persistenten Zustand einer Zustellung an den Empfänger.

Attribut	Erklärung
deliveryId	Unique Id einer HTTP-Zustellung. Die Verwendung als Idempotenzschlüssel ist im Abschnitt "Idempotenz" geregelt.
messageId	Verweist auf die <code>OriginalMessage</code> und korreliert deren Zustellung mit der Zustellung der zugehörigen <code>CanonicalMessage</code> .
deliveryType	Unterscheidet die Arten der Zustellungen.
deliverySequence	Fortlaufende Nummer, mit der der <code>DeliveryService</code> eine strikte Zustellreihenfolge erzwingt. Ein späterer Datensatz darf einen früheren unbestätigten Datensatz nicht überholen.
status	Aktueller Lebenszykluszustand der Zustellung.
lastHttpStatus	Der zuletzt empfangene HTTP-Status.
lastErrorType	Maschinenlesbare Version des zuletzt aufgetretenen HTTP-Zustellungsfehlers.
lastErrorMessage	Kurzbeschreibung des zuletzt aufgetretenen HTTP-Zustellungsfehlers.
attemptCount	Anzahl der im aktuellen Zustellzyklus gestarteten HTTP-Zustellversuche einschließlich des ersten Zustellversuchs. Der Wert ist vor dem ersten Versuch 0 und wird unmittelbar vor jedem neuen HTTP-Zustellversuch um 1 erhöht. Er bestimmt das Wiederholungsverhalten und wird nach einem erfolgreichen HealthCheck oder einer administrativen Freigabe auf 0 zurückgesetzt.
nextAttemptTime	Frühester Zeitpunkt, zu dem eine Zustellung im Wiederholungszustand erneut versucht werden darf.

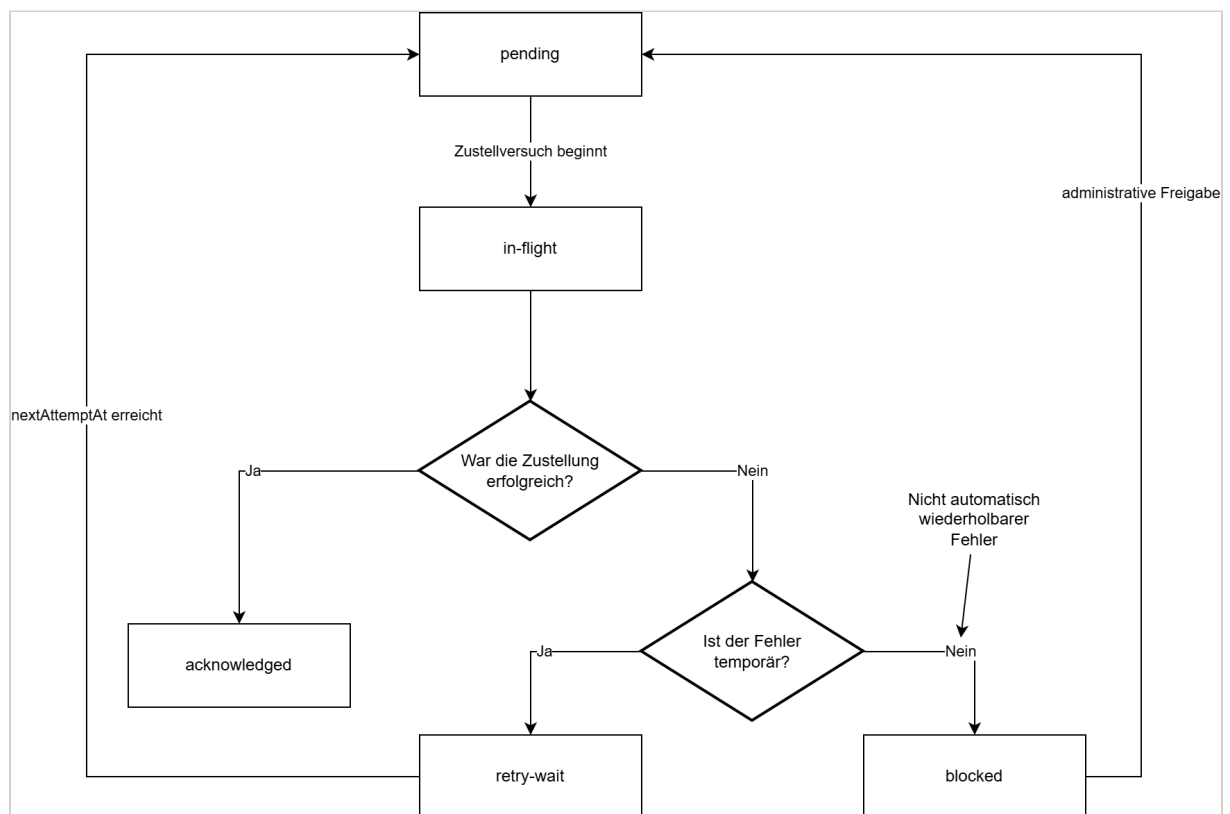
Attribut	Erklärung
<code>lastAttemptAt</code>	Zeitpunkt, zu dem der aktuelle beziehungsweise letzte HTTP-Zustellversuch gestartet wurde. Derselbe Wert wird für den aktuellen Versuch als <code>sentAt</code> in die HTTP-Message übernommen.
<code>acknowledgedAt</code>	Zeitpunkt, zu dem der Empfänger den persistenten Empfang bestätigt hat. Der Wert bleibt bis zur Bestätigung leer.
<code>createdAt</code>	Zeitpunkt, zu dem der Zustelldatensatz erzeugt wurde.

Zulässige Werte für `deliveryType` :

Wert	Bedeutung
<code>original</code>	Es handelt sich um die Zustellung einer OriginalMessage.
<code>canonical</code>	Es handelt sich um die Zustellung einer CanonicalMessage.

Zulässige Werte für `status` :

Wert	Bedeutung
<code>pending</code>	Der DeliveryRecord wurde persistent erzeugt und wartet auf seinen ersten oder nächsten Zustellversuch.
<code>in-flight</code>	Der DeliveryService verarbeitet den Datensatz aktuell. Die HTTP-Anfrage wurde gestartet oder wartet noch auf die Antwort des Overwatch-Service.
<code>retry-wait</code>	Der letzte Zustellversuch ist fehlgeschlagen. Der Datensatz bleibt persistent gespeichert und wartet bis zum Zeitpunkt <code>nextAttemptAt</code> auf den nächsten Versuch.
<code>acknowledged</code>	Der Empfänger hat die Zustellung nach persistenter Speicherung erfolgreich bestätigt. <code>acknowledgedAt</code> ist gesetzt; weitere Zustellversuche sind nicht erforderlich.
<code>blocked</code>	Der Empfänger hat einen nicht automatisch wiederholbaren Fehler zurückgegeben. Der DeliveryRecord bleibt persistent gespeichert, wird nicht automatisch zugestellt und blockiert alle späteren <code>deliverySequence</code> -Werte, bis er administrativ auf <code>pending</code> zurückgesetzt wird.



ReceiverState

Attribut	Erklärung
<code>receiverId</code>	Identifiziert den Empfänger.
<code>availabilityStatus</code>	Status der Verfügbarkeit.
<code>unavailableSince</code>	Zeitpunkt, ab dem der Empfänger als nicht verfügbar gekennzeichnet wurde.
<code>lastHealthCheckAt</code>	Zeitpunkt, zu dem der letzte HealthCheck ausgeführt wurde.
<code>nextHealthCheckAt</code>	Zeitpunkt, zu dem der nächste HealthCheck ausgeführt werden soll.
<code>lastHealthCheckStatus</code>	Letzter Status eines HealthChecks

Zulässige Werte für `availabilityStatus` :

Wert	Bedeutung
available	Die HTTP-Zustellung an den Empfänger funktioniert regulär.
unavailable	Die HTTP-Zustellung an den Empfänger funktioniert nicht. Zustellversuche werden eingestellt und es werden nur noch HealthChecks durchgeführt.

ErrorCase

`ErrorCase` speichert die Metadaten eines Fehlers im Verarbeitungspfad für transformierte Telemetrie. Der `ErrorCase` wird vom Process-Worker erzeugt und persistent gespeichert.

Attribut	Erklärung
<code>errorId</code>	Eindeutiger Identifikator des Fehlerfalls und des Diagnosepakets.
<code>messageId</code>	Verweist auf die vollständig gespeicherte fehlerhafte <code>OriginalMessage</code> .
<code>processingStage</code>	Verarbeitungsschritt, in dem der Fehler auftrat.
<code>errorType</code>	Maschinenlesbare Klassifikation des Fehlers.
<code>errorMessage</code>	Menschenlesbare Fehlerbeschreibung.
<code>occurredAt</code>	Zeitpunkt des Verarbeitungsfehlers.
<code>diagnosticPackageLocation</code>	Speicherort oder Abrufschlüssel des vollständigen Diagnosepakets
<code>status</code>	Bearbeitungszustand der Fehlerbehandlung.
<code>completedAt</code>	Zeitpunkt, zu dem Logeintrag und Diagnosepaket vollständig erstellt wurden.
<code>lastAttemptAt</code>	Zeitpunkt, zu dem zuletzt versucht wurde, die Fehlerbehandlung durchzuführen.
<code>nextAttemptAt</code>	Zeitpunkt, zu dem die Behandlung dieses <code>ErrorCase</code> erneut durchgeführt werden soll.

Attribut	Erklärung
attemptCount	Anzahl der gestarteten processErrorCase -Versuche. Der Wert ist vor dem ersten Versuch 0 und wird zu Beginn jedes Diagnoseversuchs um 1 erhöht.

Zulässige Werte für processingStage :

Wert	Bedeutung
identification	Der Fehler trat während der Identifikation der Drohne bzw. der Zuordnung von sourceId zu deviceId auf.
raw-data-validation	Die Telemetrierohdaten konnten nicht validiert werden.
transformation	Die Ausführung der JSON-Transformation der Telemetrierohdaten ist fehlgeschlagen.
transformed-data-validation	Die transformierte Telemetrie konnte nicht validiert werden (entspricht nicht dem erwarteten Ergebnis).
internal-processing	Unerwarteter technischer Fehler außerhalb der Verarbeitungsschritte für die Transformation der Telemetriedaten, beispielsweise ein Programmfehler, eine nicht verfügbare Datei o. ä.

Zulässige Werte für status :

status	Bedeutung
pending	Der ErrorCase ist persistent gespeichert. Der Process-Worker hat die Erzeugung von Logeintrag und Diagnosepaket noch nicht begonnen.
processing	Der Process-Worker erzeugt mit Hilfe des ErrorHandling Packages aktuell den error.log-Eintrag und das Diagnosepaket.

status	Bedeutung
retry-wait	Eine erforderliche Diagnoseoperation ist fehlgeschlagen, z. B. das Schreiben von error.log, das Erzeugen oder Prüfen des ZIP-Archivs oder dessen Ablage im Diagnostic Repository. Der Process-Worker führt später einen erneuten Versuch aus.
completed	Der Process-Worker hat Logeintrag und Diagnosepaket vollständig erstellt und geprüft.

SourceProcessingState

SourceProcessingState verfolgt die aktuelle Zuordnung zwischen Quelle und Drohne.

SourceProcessingState ist über einen Neustart hinweg persistent. Die Zuordnung von sourceId zu deviceId wird beim Startup jedoch zurückgesetzt, da sie nicht als vertrauenswürdiger neuer Kontext verwendet werden darf.

Attribut	Erklärung
sourceId	Identifiziert den Controller beziehungsweise die veröffentlichende Quelle, deren aktuelle Zuordnung verfolgt wird.
identificationStatus	Gibt an, ob die Quelle unidentified, oder identified ist.
deviceId	Identifiziert die aktuell zugeordnete registrierte Drohne. Der Wert bleibt leer, solange die Quelle nicht identifiziert ist.
lastMessageAt	Zeitpunkt der letzten Nachricht der Quelle. Der Wert unterstützt die konfigurierte inaktivitätsbasierte Regel zur erneuten Identifikation.

Zulässige Werte für identificationStatus :

Wert	Bedeutung
unidentified	Der Quelle ist aktuell keine Drohne zugeordnet.

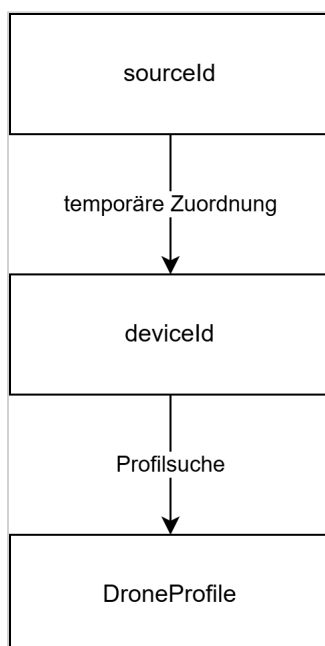
Wert	Bedeutung
identified	Der Adapter hat für den aktuellen Quellkontext eine <code>deviceId</code> ausgewählt. Spätere Nachrichten derselben Quelle dürfen das aufgelöste Profil verwenden, auch wenn sie die Drohne nicht selbstständig identifizieren.

Bei jeder neuen `OriginalMessage`, die der `Process-Worker` aus dem `inputBuffer` übernimmt, prüft er vor der Aktualisierung von `lastMessageAt`, ob eine bestehende Zuordnung weiterhin vertrauenswürdig ist. Ist `identificationStatus = identified` und überschreitet die Differenz zwischen `OriginalMessage.receivedAt` und dem bisherigen `SourceProcessingState.lastMessageAt` das konfigurierte Inaktivitätsintervall, verwirft der `Process-Worker` die bisherige Zuordnung. Er setzt `identificationStatus = unidentified` und `deviceId = null`.

Widerspricht die aktuelle Nachricht dem bisher aktiven Drohnenprofil, wird die Zuordnung ebenfalls auf `unidentified / deviceId = null` zurückgesetzt und die aktuelle Message erneut zur Identifikation verwendet. Anschließend setzt der `Process-Worker` `SourceProcessingState.lastMessageAt = OriginalMessage.receivedAt` und speichert den Zustand.

Nachrichten, die nachträglich aus dem `clipboardBuffer` abgearbeitet werden, aktualisieren `lastMessageAt` nicht. Dadurch kann ein älterer `receivedAt`-Wert den bereits gespeicherten Zeitpunkt einer später eingegangenen Nachricht nicht überschreiten.

Die Zuordnung `sourceId` zu `deviceId` ist temporär:



Eine `sourceId` darf niemals als dauerhafte Drohnenidentität behandelt werden.

ErrorHandling Package

Das `ErrorHandling` Package enthält alle Funktionen zum Management des `error.log` und der Diagnosepackages im Diagnostic Repository. Seine Funktionen werden durch den Process-Worker genutzt. Es stellt Funktionen bereit für:

- die Erstellung eines strukturierten Eintrags im `error.log`
- die Erzeugung eines Diagnosepakets im Diagnostic Repository
- die Ablage der verwendeten Schemata als Snapshot im Diagnosepaket
- die Überprüfung des Diagnosepakets (Prüfsumme und Vollständigkeit)

Inhalt des error.log

`error.log` wird als Text-Datei geführt. Jede Zeile enthält einen eigenständigen strukturierten Logdatensatz. Das Schreiben von Einträgen in `error.log` erfolgt mit At-least-once-Semantik.

Das Anhängen eines Logeintrages und die persistente Aktualisierung des zugehörigen `ErrorCase` können nicht gemeinsam in einer atomaren Transaktion erfolgen. Wird der Adapter zwischen diesen beiden Operationen unterbrochen, kann bei der Wiederholung ein weiterer Logeintrag mit derselben `errorId` entstehen. Mehrere Logeinträge mit derselben `errorId` beschreiben denselben logischen Fehlerfall. Anwendungen, die das `error.log` auswerten, müssen diese Einträge anhand von `errorId` korrelieren bzw. deduplizieren.

Attribut	Bedeutung
<code>timestamp</code>	Zeitpunkt, zu dem der Logeintrag geschrieben wurde.
<code>level</code>	Log-Level
<code>component</code>	Komponente, in der der Verarbeitungsfehler auftrat.
<code>errorId</code>	Identifikator des Fehlerfalls und Diagnosepakets.
<code>messageId</code>	Identifikator der fehlerhaften Originalnachricht.
<code>sourceId</code>	Controller beziehungsweise Quelle der Nachricht.
<code>processingStage</code>	Verarbeitungsschritt, in dem der Fehler auftrat.

Attribut	Bedeutung
<code>errorType</code>	Maschinenlesbare Fehlerklassifikation.
<code>errorMessage</code>	Menschenlesbare Fehlerbeschreibung.
<code>occurredAt</code>	Zeitpunkt des ursprünglichen Verarbeitungsfehlers.
<code>diagnosticPackageLocation</code>	Vollständiger Dateipfad des ZIP-Archivs.

Die vollständige Nutzlast und die vollständigen Schemata werden nicht in `error.log` geschrieben. Sie befinden sich im Diagnosepaket.

Diagnosepaket

Das Diagnosepaket legt alle für Reproduktion und Analyse benötigten Daten gemeinsam unter derselben `errorId` ab. Die folgende Struktur ist die interne Struktur eines ZIP-Archivs (jedes Diagnosepaket ist ein ZIP-Archiv).

```

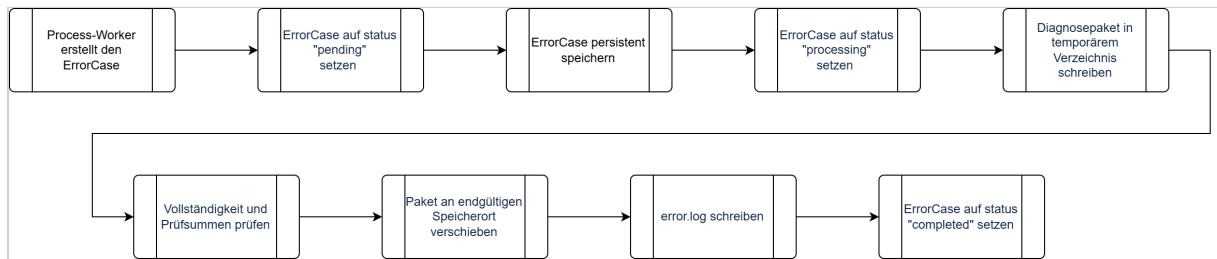
errors/
├─ {errorId}/
│   ├─ manifest.json
│   ├─ error.json
│   ├─ original-message.json
│   ├─ mqtt-metadata.json
│   ├─ identification/
│   │   ├─ profiles/
│   │   ├─ message-definitions/
│   │   └─ schemas/
│   ├─ validation/
│   │   ├─ raw-schema.json
│   │   ├─ raw-validation-result.json
│   │   ├─ canonical-schema.json
│   │   └─ canonical-validation-result.json
│   └─ transformation/
│       ├─ transformation.jsonata
│       └─ transformation-metadata.json

```

Nicht vorhandene Zwischenergebnisse werden ausgelassen. Die tatsächlich verwendeten Profile, Schemata und Transformationen werden als Snapshots abgelegt, weil sich Repository-Inhalte später ändern können.

`manifest.json` enthält mindestens `errorId`, `messageId`, Erstellungszeitpunkt sowie für jede Datei Pfad, Inhaltstyp, Größe und Prüfsumme.

Schreibablauf



Kann nicht sichergestellt werden, ob der Logeintrag bereits geschrieben wurde, darf er erneut geschrieben werden. Dabei gilt die im Abschnitt **Inhalt des `error.log`** definierte At-least-once-Semantik.

Fehlerklassifikation

Persistierung der MQTT-Message schlägt fehl:

- MQTT-PUBACK nicht freigeben und dadurch Backpressure anwenden,
- persistMqttMessage gemäß dem definierten Retry-Schema erneut ausführen,
- nach insgesamt 5 erfolglosen Versuchen die MQTT-Verbindung trennen, ohne die persistente Session oder das Abonnement zu löschen.

Drohne kann noch nicht identifiziert werden:

- Originalzustellung läuft weiter,
- Verarbeitungspfad für transformierte Telemetrie schreibt in den `clipboardBuffer`.

Drohne bleibt beim Herunterfahren oder nach einem unsicheren Neustart nicht identifiziert:

- Originalzustellung bleibt garantiert,
- Verarbeitungspfad für transformierte Telemetrie wird abgebrochen und protokolliert.

Validierung oder Transformation schlägt fehl:

- Originalzustellung bleibt garantiert,
- Process-Worker erzeugt und persistiert den `ErrorCase`,
- Process-Worker erzeugt das Diagnosepaket,
- Process-Worker erzeugt den Eintrag im `error.log`.

Empfänger ist nicht verfügbar:

- `DeliveryRecords` bleiben ausstehend,
- Healthchecks werden fortgesetzt,
- Zustellung wird nach der Wiederherstellung fortgesetzt.

Adapter stürzt ab:

- akzeptierte Telemetrie und ausstehende Zustellungen werden aus der Datenbank wiederhergestellt

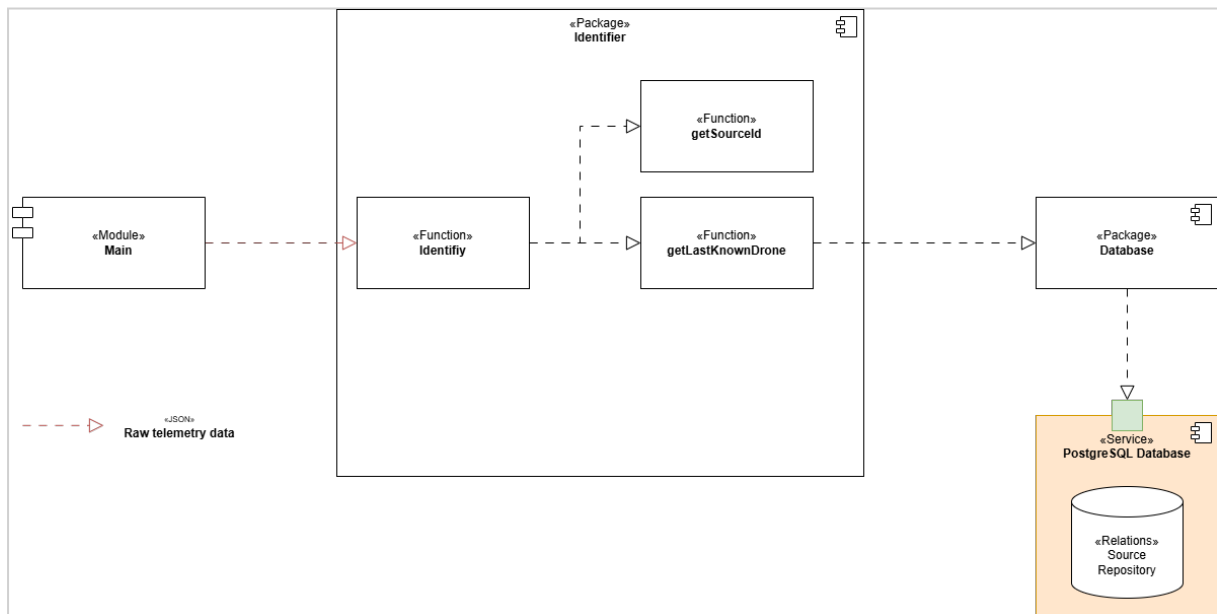
Persistenter Speicher ist voll oder nicht verfügbar:

- Akzeptieren und Bestätigen von MQTT-Nachrichten stoppen

Empfänger bestätigt Speicherung, aber HTTP-Antwort geht verloren:

- Adapter wiederholt dieselbe deliveryId
- Empfänger dedupliziert

Identifizier-Package



Das Identifier-Package erhält die Telemetrierohdaten als Eingabe. In der Funktion `getSourceId` extrahiert es die `sourceId` aus dem Topic, d. h. aus dem Topic `raw/source/73da918/thing/product/TH7825301867/osd` wird der Wert `73da918` entnommen. Dadurch kann der Controller / das Gateway identifiziert werden.

In einem nächsten Schritt wird die Funktion `getLastKnownDrone` mit der `sourceId` aufgerufen. Diese Funktion holt die Message Definitionen für die Drohne, welche zuletzt mit dem Controller `73da918` geflogen wurde, aus der Datenbank.

Zu jeder Message, welche ein Drohnentyp senden kann, muss eine Message Definition existieren. Nun wird versucht, die vorliegende Message zu erkennen. Dazu müssen alle gefundenen Message Definitionen auf die Message angewandt werden, bis ein Match vorliegt. Eine Message Definition enthält folgende Attribute (hier als JSON dargestellt):

Message Definition	
1	{
2	"definitionId": 16,
3	"topic": {
4	"normalizedTopic": "thing/product/state",
5	"topicPattern": "^thing/product/[A-Z0-9]{12}/state\$",
6	},
7	"payloadSelectors": [
8	{
9	"pointer": "/method",
10	"operator": "equals",
11	"value": "target_detect_result_report"
12	}
	}

13	],
14	"rawSchema": 42,
15	"normalizedSchema": 34,
16	"transformationSchema": 47
17	}

Attribut	Bedeutung
definition Id	Numerische Id des Drohnenprofils. Mit Hilfe dieser Id kann das Drohnenprofil aus der Datenbank geholt werden.
normalized Topic	Ein Topic, aus dem alle veränderlichen Bestandteile (Seriennummer, Modellnummer, ...) entfernt wurden. Er dient zur Identifikation von Message Definitions, die zu diesem Topic passen können.
topic Pattern	Ein regulärer Ausdruck, mit dessen Hilfe aus einem Topic (z. B. <code>thing/product/TH7825301867/state</code>) der normalisierte Topic <code>thing/product/state</code> kreiert werden kann.
payload Selector	Payload Selectors sind Regeln, die zur einfachen Identifizierung einer Nachricht anhand der Payload dienen. Im obigen Beispiel bedeutet <code>/method equals target_detect_result_report</code> , dass in der Payload der Message das Attribut <code>method</code> auf erster Ebene mit dem Wert <code>target_detect_result_report</code> vorkommen muss. Wird diese Kombination vorgefunden, gilt die Nachricht als identifiziert.
rawSchema	Numerische Id des JSON-Schemas zur Identifizierung und Validierung der Telemetrierohdaten der Message.
normalized Schema	Numerische Id des JSON-Schemas zur Validierung der transformierten Telemetriedaten der Message.
transformationSchema	Numerische Id des JSONata-Schemas, das zur Transformation der Telemetrierohdaten der Message genutzt werden soll.

Zuerst wird versucht, mit Hilfe der Payload Selectors die Nachricht zu identifizieren. Passen die Message und ein Payload Selector zusammen, wird die Nachricht zusätzlich noch mit dem Raw Schema der betreffenden Message Definition validiert. Liegt auch hier eine Übereinstimmung vor, wurde die Nachricht eindeutig erkannt und somit auch der Drohnentyp.

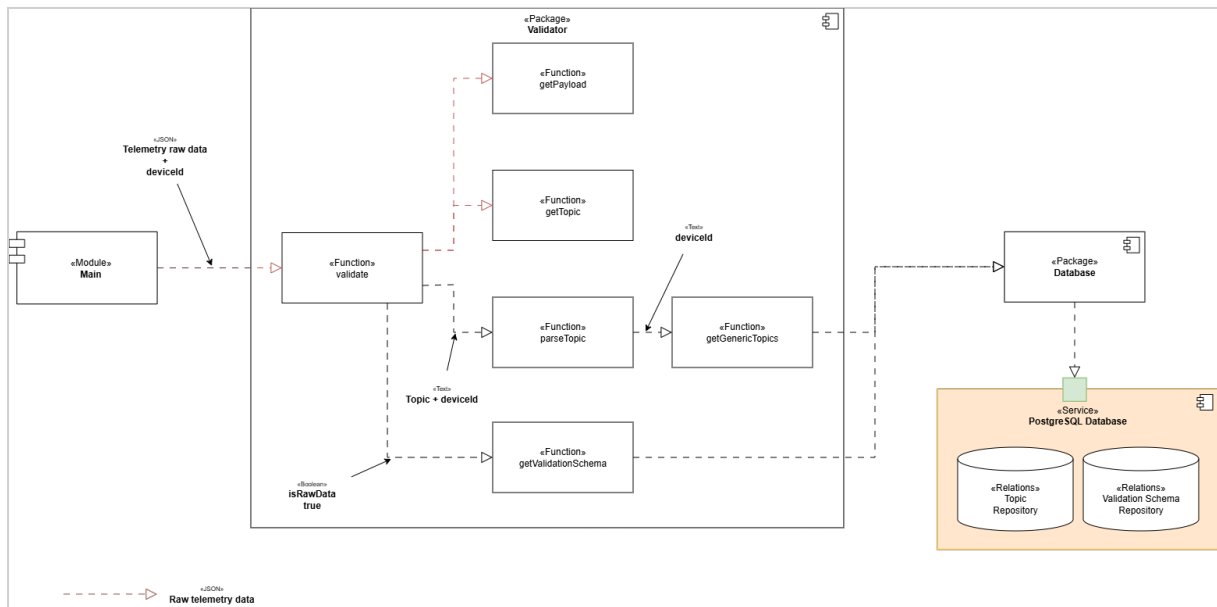
Konnte mit Hilfe der Payload Selectors keine Übereinstimmung erzielt werden, bedeutet dies, dass der Controller einen anderen Drohnentyp fliegt.

Die Identifizierung anhand des zuletzt benutzten Drohnentyps ist somit fehlgeschlagen. Es wird nun versucht, mit Hilfe der Topic Pattern aller registrierten Drohnentypen eine Erkennung der Nachricht durchzuführen.

Validator-Package

Die Aufgabe des Validator-Packages ist es, Telemetriedaten auf ihre Korrektheit zu überprüfen. Es kann sowohl die Rohdaten als auch die transformierten Daten validieren.

Validierung der Telemetrierohdaten



Bei der Validierung der Rohdaten wird die Funktion `validate` direkt mit den Telemetrierohdaten versorgt. Diese nutzt die Funktion `getPayload`, um die Payload (Daten, die in der MQTT-Message transportiert werden) zu erhalten.

Mit `getTopic` trennt `validate` den Original-Topic aus den Rohdaten heraus. D. h. aus `/raw/source/73da918/thing/product/TH7825301867/ods` wird `thing/product/TH7825301867/ods`.

`parseTopic` ist dafür zuständig, veränderliche Bestandteile, wie z. B. Seriennummern aus dem Original-Topic herauszutrennen. Dafür benutzt `parseTopic` sogenannte "generische Topics", die es mit Hilfe von `getGenericTopics` aus dem Topic Repository holt. Zur Identifizierung der generischen Topics wird die `deviceId` benutzt. Ein generischer Topic ist ein regulärer Ausdruck, der es ermöglicht, veränderliche Bestandteile aus dem Topic zu entfernen. Beispielsweise könnte `thing/product/[A-Z]+[\d]+/osd` der generische Topic sein, der aus `thing/product/TH7825301867/ods` den normalisierten Topic `thing/product/osd` macht. Dieser normalisierte Topic `thing/product/osd` dient im Validation Schema Repository zur Identifikation des Validierungsschemas.

`getValidationSchema` nutzt den normalisierten Topic `thing/product/osd`, um das Validierungsschema für die Telemetrierohdaten aus dem Validation Schema Repository zu holen.

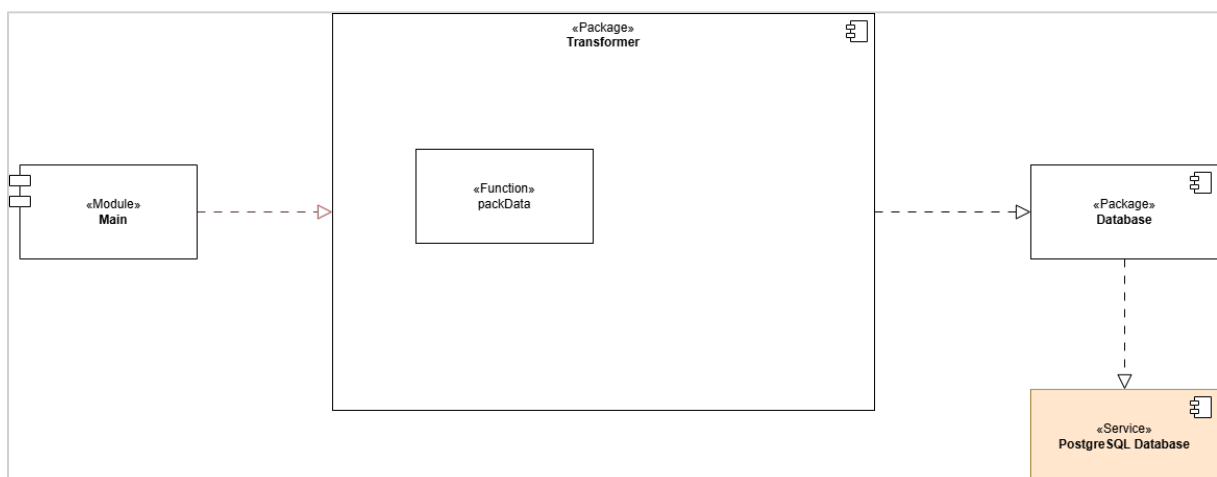
Beim Aufruf von `getValidationSchema` wird zusätzlich zum normalisierten Topic mit `isRawData = true` angegeben, dass das Validierungsschema für die Telemetrierohdaten geholt werden soll.

Die Funktion `validate` kann unter Zuhilfenahme des Validierungsschemas prüfen, ob die Telemetrierohdaten korrekt sind.

Validierung der transformierten Telemetriedaten

Die Validierung der transformierten Telemetriedaten verläuft im Wesentlichen genauso wie die Validierung der Rohdaten. Einziger Unterschied ist, dass beim Aufruf von `getValidationSchema` durch die Angabe von `isRawData = false` das Validierungsschema für die transformierten Telemetriedaten aus der Datenbank geholt wird.

Transformer-Package



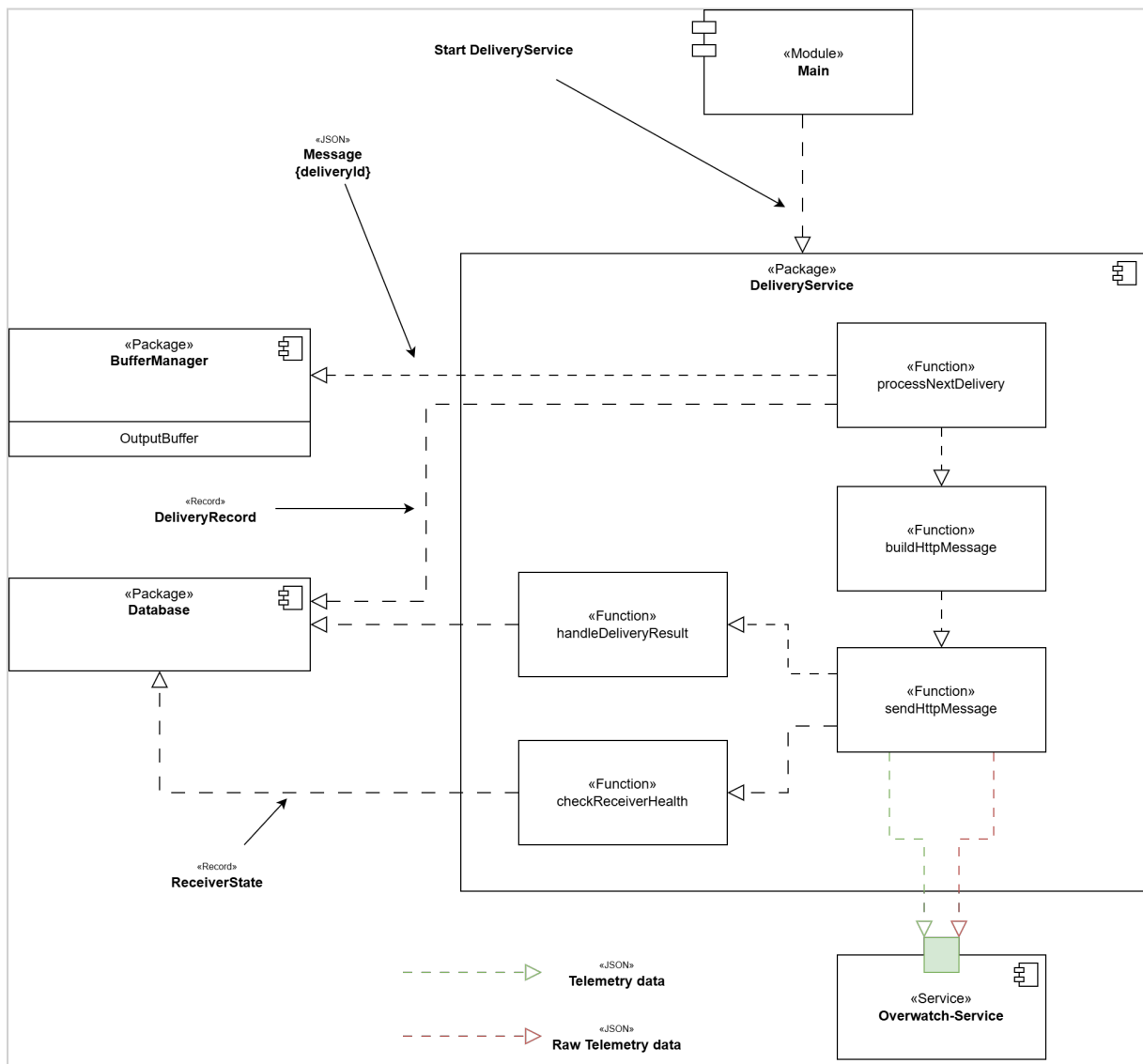
TODO: Funktion `packMessage` einplanen. Dabei wird der Topic `/raw/source/{sourceId}/thing/product/{serial}/osd` in `/normalized/device/{deviceId}/thing/product/{serial}/osd` umgewandelt und zusammen mit der Payload in eine Nachricht gepackt.

DeliveryService-Package

Das `DeliveryService`-Package ist für die persistente At-least-once-Zustellung von Telemetrierothdaten und transformierter Telemetrie an den Overwatch-Service verantwortlich. Die zuzustellenden Daten selbst werden nicht im `DeliveryService` gehalten. Der `outputBuffer` enthält ausschließlich `deliveryId`-Werte und bildet die globale Zustellreihenfolge ab. Jeder `DeliveryRecord` mit `status != acknowledged` befindet sich genau einmal im `outputBuffer`. Die Reihenfolge entspricht `deliverySequence` aufsteigend. Daher ist der Datensatz am Kopf des `outputBuffer` immer der führende unbestätigte `DeliveryRecord`. Der persistente `DeliveryRecord` bleibt die maßgebliche Quelle für den Zustellzustand. Der `outputBuffer` bestimmt ausschließlich, welcher `DeliveryRecord` als nächster betrachtet werden darf.

Ein `DeliveryRecord` wird erst aus dem `outputBuffer` entfernt, nachdem sein Status `acknowledged` gesetzt wurde.

`DeliveryRecord`-Datensätze für Telemetrierothdaten erhalten ihre `deliverySequence` bei der Annahme der MQTT-Message. `DeliveryRecords` für transformierte Telemetrie erhalten ihre `deliverySequence`, sobald die Verarbeitung für transformierte Telemetrie abgeschlossen wurde. Alle `DeliveryRecords` verwenden eine gemeinsame globale `deliverySequence`. Dadurch wird eine globale Zustellreihenfolge für Original- und transformierte Telemetrie gewährleistet.



Funktion	Aufgabe
processNextDelivery	Koordiniert die Verarbeitung des führenden DeliveryRecord am Kopf des outputBuffer . Die Funktion prüft den ReceiverState und den persistenten Zustand dieses DeliveryRecord , lädt bei einer zulässigen Zustellung die benötigten persistenten Daten und startet genau einen Zustellversuch.
buildHttpMessage	Erzeugt aus dem DeliveryRecord und den referenzierten Telemetriedaten die an den Overwatch-Service zu sendende HTTP-Message. Der Aufbau hängt vom deliveryType ab.

Funktion	Aufgabe
<code>sendHttpMessage</code>	Führt genau einen HTTP-POST-Zustellversuch an den Overwatch-Service aus und liefert das Ergebnis des Versuchs zurück. Die Funktion führt selbst keine Wiederholungsversuche aus.
<code>handleDeliveryResult</code>	Bewertet das Ergebnis eines Zustellversuchs und aktualisiert den persistenten <code>DeliveryRecord</code> . Abhängig vom Ergebnis wird die Zustellung bestätigt, für eine Wiederholung vorgemerkt oder blockiert. Bei länger anhaltender Nichterreichbarkeit wird zusätzlich der <code>ReceiverState</code> aktualisiert.
<code>checkReceiverHealth</code>	Prüft die Erreichbarkeit des Overwatch-Service, solange <code>ReceiverState.availabilityStatus = unavailable</code> ist. Bei erfolgreicher Prüfung wird die reguläre Zustellung wieder freigegeben.

Der `BufferManager` stellt dem `DeliveryService` die `deliveryId` am Kopf des `outputBuffers` bereit. Der zugehörige persistente `DeliveryRecord` bestimmt, ob aktuell eine Zustellung ausgeführt werden darf. Ein führender `DeliveryRecord` mit status `in-flight`, `retry-wait` oder `blocked` verbleibt am Kopf des `outputBuffer` und blockiert dadurch sämtliche spätere `DeliveryRecord`-Datensätze unabhängig von deren `deliveryType`.

Das Database Package stellt dem `DeliveryService` den persistenten `DeliveryRecord`, die referenzierten Telemetriedaten und den `ReceiverState` zur Verfügung.

Die Erzeugung des `DeliveryRecords` für die Rohdaten ist unabhängig von der Transformation. Die spätere HTTP-Zustellung von Telemetrierohdaten und transformierter Telemetrie unterliegt jedoch der gemeinsamen globalen `deliverySequence`.

Daher gilt:

- Originalnachricht N kann vor der transformierten Nachricht N eintreffen.
- Originalnachricht N+1 kann vor der transformierten Nachricht N eintreffen.

Der Overwatch-Service verbindet beide Darstellungen über das Attribut `messageId`.

Normative Zustandsübergänge

Die folgende Tabelle definiert die maßgeblichen Zustandsübergänge. Ablaufdiagramme und Funktionsbeschreibungen konkretisieren diese Regeln, dürfen ihnen jedoch nicht widersprechen.

Ereignis	Bedingung	Persistente Änderung	Folge
Zustellversuch starten	führender DeliveryRecord hat status = pending, ReceiverState.availabilityStatus = available	DeliveryRecord.status = in-flight, DeliveryRecord.attemptCount++, DeliveryRecord.lastAttemptAt = now, DeliveryRecord.nextAttemptAt = null	genau einen HTTP-Zustellversuch starten
HTTP 2xx-Response	in-flight	DeliveryRecord.status = acknowledged, DeliveryRecord.acknowledgedAt = now	nach Commit Kopf aus outputBuffer entfernen ggf. nächsten DeliveryRecord verarbeiten.
temporärer Fehler	attemptCount < 13	DeliveryRecord.status = retry-wait, DeliveryRecord.nextAttemptAt gemäß Retry-Schema	DeliveryRecord bleibt am Kopf und der Retry-Timer muss aktualisiert werden
temporärer Fehler	attemptCount = 13	DeliveryRecord.status = pending, DeliveryRecord.nextAttemptAt = null ReceiverState.availabilityStatus = unavailable, ReceiverState.unavailableSince = now, ReceiverState.nextHealthCheckAt = Zeitpunkt des nächsten HealthChecks	DeliveryRecord bleibt am Kopf des outputBuffer. die reguläre Zustellung wird ausgesetzt.

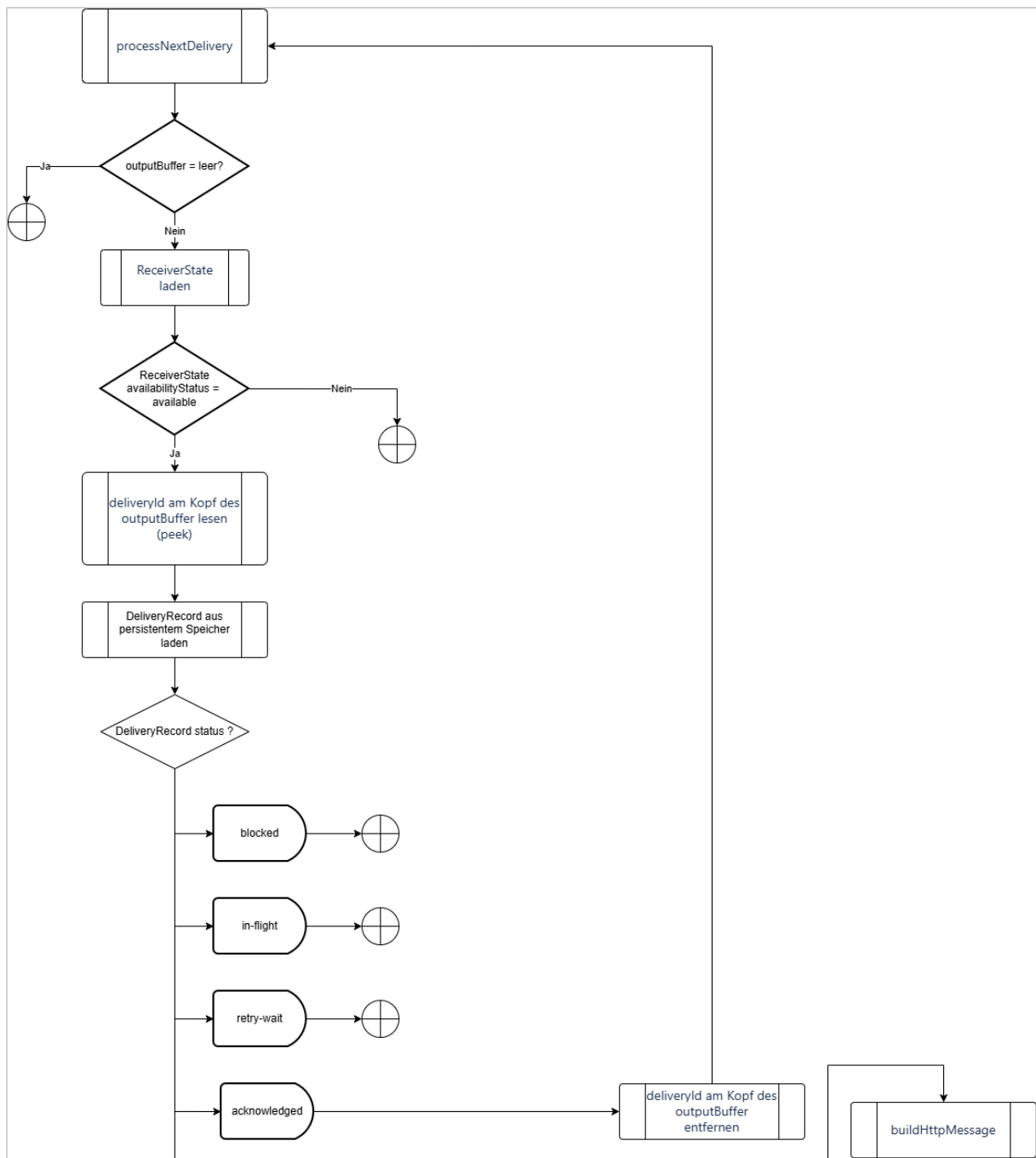
Ereignis	Bedingung	Persistente Änderung	Folge
nicht automatisch wiederholbare Fehler	HTTP 3xx bzw. nicht temporärer HTTP 4xx Fehler	<code>DeliveryRecord.status = blocked</code>	<code>DeliveryRecord</code> bleibt am Kopf des <code>outputBuffer</code> und blockiert spätere <code>DeliveryRecord</code> -Datensätze
Retry-Zeitpunkt erreicht	führender <code>DeliveryRecord.status = retry-wait</code>	<code>DeliveryRecord.status = pending</code> , <code>DeliveryRecord.nextAttemptAt = null</code>	<code>DeliveryService.processNextDelivery</code> aufrufen
administrative Freigabe	<code>DeliveryRecord.status = blocked</code>	<code>DeliveryRecord.status = pending</code> , <code>DeliveryRecord.nextAttemptAt = null</code> , <code>DeliveryRecord.attemptCount = 0</code>	<code>DeliveryService.processNextDelivery</code> aufrufen
HealthCheck erfolgreich	<code>ReceiverState = unavailable</code>	<code>ReceiverState.availabilityStatus = available</code> , <code>ReceiverState.unavailableSince = null</code> , <code>ReceiverState.nextHealthCheckAt = null</code> , führender <code>DeliveryRecord.attemptCount = 0</code>	<code>DeliveryService.processNextDelivery</code> aufrufen
HealthCheck fehlgeschlagen	<code>ReceiverState = unavailable</code>	<code>ReceiverState.availabilityStatus = unavailable</code> , <code>ReceiverState.nextHealthCheckAt = Zeitpunkt des nächsten HealthChecks</code>	keine reguläre Zustellung

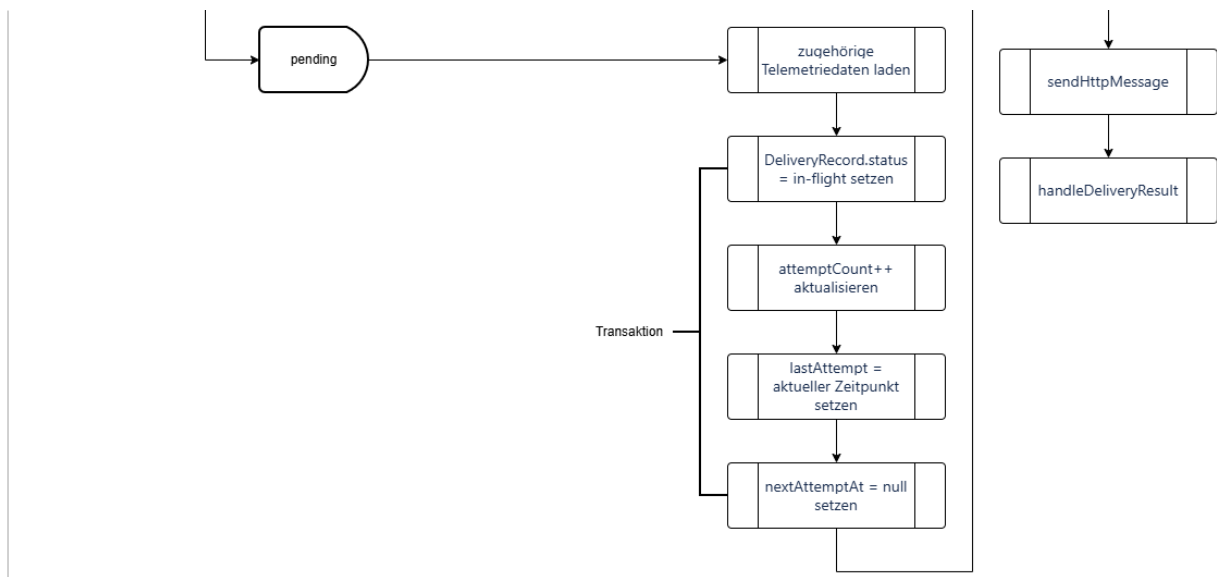
Wesentliche Funktionen

processNextDelivery

`processNextDelivery` ist die zentrale koordinierende Funktion des `DeliveryService`. Sie wird ausgeführt, wenn eine reguläre Zustellung durchgeführt werden kann und ein `DeliveryRecord` zur Verarbeitung ansteht.

Der Ablauf ist:





Der `outputBuffer` bestimmt die globale Reihenfolge der Zustellungen. Sein Kopf entspricht aufgrund der Buffer-Invariante stets dem noch nicht bestätigten `DeliveryRecord` mit der kleinsten `deliverySequence`. Der persistente `DeliveryRecord` bestimmt dagegen den aktuellen Lebenszykluszustand der Zustellung. Nur ein führender `DeliveryRecord` mit `status = pending` darf einen neuen HTTP-Zustellversuch starten. `retry-wait`, `blocked` und `in-flight` verbleiben im Kopf des `outputBuffer` und verhindern dadurch, dass ein späterer `DeliveryRecord` den führenden Datensatz überholt.

buildHttpMessage

`buildHttpMessage` erzeugt aus dem persistenten `DeliveryRecord` und den über `messageId` referenzierten Telemetriedaten die HTTP-Message für den Overwatch-Service.

Die Funktion unterscheidet anhand von `DeliveryRecord.deliveryType` zwischen:

- `original` – Zustellung der Telemetrierohdaten;
- `canonical` – Zustellung der transformierten Telemetriedaten.

Die folgende Tabelle beschreibt die an den Overwatch-Service gesendeten HTTP-Messages.

HTTP-Message für Telemetrierohdaten

Attribut	Erklärung
<code>deliveryId</code>	Idempotenzschlüssel dieser Zustellung an den Empfänger (siehe Abschnitt "Idempotenz").
<code>deliveryType</code>	Fester Wert <code>original</code> .
<code>deliverySequence</code>	Fortlaufende Zahl, welche eine eindeutige Reihenfolge der DeliveryRecords garantiert.
<code>messageId</code>	Korreliert die Zustellung der Telemetrierohdaten mit der Zustellung der transformierten Telemetrie.
<code>enrichedTopic</code>	Überträgt den Quellnamensraum und den verlustfrei codierten ursprünglichen MQTT-Topic.
<code>payload</code>	Enthält die Telemetrierohdaten.
<code>createdAt</code>	Zeitpunkt, zu dem dieser Record erstellt wurde.
<code>sentAt</code>	Zeitpunkt, zu dem dieser konkrete HTTP-Zustellversuch gestartet wurde. Der Wert wird nicht zur Duplikaterkennung herangezogen. Der Wert entspricht <code>DeliveryRecord.lastAttemptAt</code> des aktuellen Zustellversuchs.

HTTP-Message für transformierte Telemetrie

Attribut	Erklärung
<code>deliveryId</code>	Idempotenzschlüssel dieser Zustellung an den Empfänger (siehe Abschnitt "Idempotenz").
<code>deliveryType</code>	Fester Wert <code>canonical</code> .
<code>deliverySequence</code>	Fortlaufende Zahl, welche eine eindeutige Reihenfolge der <code>DeliveryRecords</code> garantiert.
<code>messageId</code>	Korreliert die Zustellung der transformierten Telemetrie mit der Zustellung der Telemetrierohdaten.
<code>deviceId</code>	Identifiziert die registrierte Drohne.
<code>payload</code>	Enthält die transformierten Telemetriedaten.
<code>createdAt</code>	Zeitpunkt, zu dem dieser Record erstellt wurde.
<code>sentAt</code>	Zeitpunkt, zu dem dieser konkrete HTTP-Zustellversuch gestartet wurde. Der Wert wird nicht zur Duplikaterkennung herangezogen. Der Wert entspricht <code>DeliveryRecord.lastAttemptAt</code> des aktuellen Zustellversuchs.

sendHttpRequest

`sendHttpRequest` führt genau einen HTTP-POST-Zustellversuch aus. Die Funktion erhält die von `buildHttpRequest` erzeugte HTTP-Message und überträgt diese an den Overwatch-Service. `sendHttpRequest` entscheidet nicht selbst über einen erneuten Zustellversuch. Das Ergebnis wird an `handleDeliveryResult` übergeben.

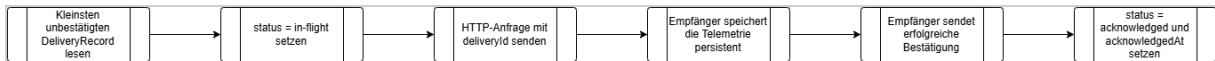
handleDeliveryResult

`handleDeliveryResult` bewertet das Ergebnis von `sendHttpRequest`, klassifiziert die HTTP-Antwort beziehungsweise den aufgetretenen Fehler und führt den in der Tabelle "Normative Zustandsübergänge" festgelegten Zustandsübergang aus. Ein `DeliveryRecord` wird nur nach einem erfolgreichen persistenten Wechsel auf `acknowledged` aus dem `outputBuffer` entfernt. Bei allen anderen Zuständen verbleibt seine `deliveryId` am Kopf des `outputBuffer`.

`handleDeliveryResult` verarbeitet genau das Ergebnis des aktuellen Zustellversuchs. Ein späterer Retry wird ausschließlich über den persistenten Zustand und den Retry-Timer des `DeliveryService` ausgelöst.

HTTP-Zustellablauf

Für die Zustellung von Telemetrydaten an den Overwatch-Service sind folgende Schritte notwendig:



Eine erfolgreiche HTTP-Antwort (HTTP 2xx) ist nur gültig, wenn der Empfänger garantiert, dass sie erst nach der persistenten Speicherung zurückgegeben wird.

Bestätigung / Acknowledgement

Der Empfänger darf erst dann eine erfolgreiche Antwort zurückgeben, wenn:

1. Die vollständige Anfrage empfangen wurde.
2. `deliveryId` auf ein Duplikat geprüft wurde.
3. Die Telemetrie persistent gespeichert wurde.
4. Die Empfängertransaktion erfolgreich abgeschlossen wurde.

Eine Bestätigung nach ausschließlicher Ablage der Nachricht in einem flüchtigen Buffer ist unzureichend.

Erfolgreiche Zustellung

Für jeden HTTP-Zustellversuch gilt ein Gesamtzeitlimit von 30 Sekunden. Es beginnt unmittelbar vor dem Verbindungsaufbau und endet, sobald die vollständige HTTP-Response empfangen wurde. Das Zeitlimit umfasst Verbindungsaufbau, gegebenenfalls TLS-Handshake, Übertragung der Anfrage und Warten auf die Antwort. Wird innerhalb dieser 30 Sekunden keine vollständige erfolgreiche Antwort empfangen, gilt der Zustellversuch als fehlgeschlagen.

Ein Zustellversuch gilt als erfolgreich, wenn der Overwatch-Service eine erfolgreiche HTTP-2xx-Antwort sendet.

Folgende Fehler gelten als temporär und werden erneut versucht:

- Netzwerkfehler,
- Überschreitung des HTTP-Timeouts,
- HTTP 408 Request Timeout,
- HTTP 429 Too Many Requests,
- HTTP 5xx

Nicht automatisch wiederholbare Fehler führen gemäß der Tabelle "Normative Zustandsübergänge" zum Zustand `blocked`. Die administrative Freigabe eines blockierten Datensatzes erfolgt ebenfalls gemäß dieser Tabelle.

Der Response-Status und eine begrenzte Fehlerbeschreibung werden für die Diagnose im `DeLiveryRecord` gespeichert.

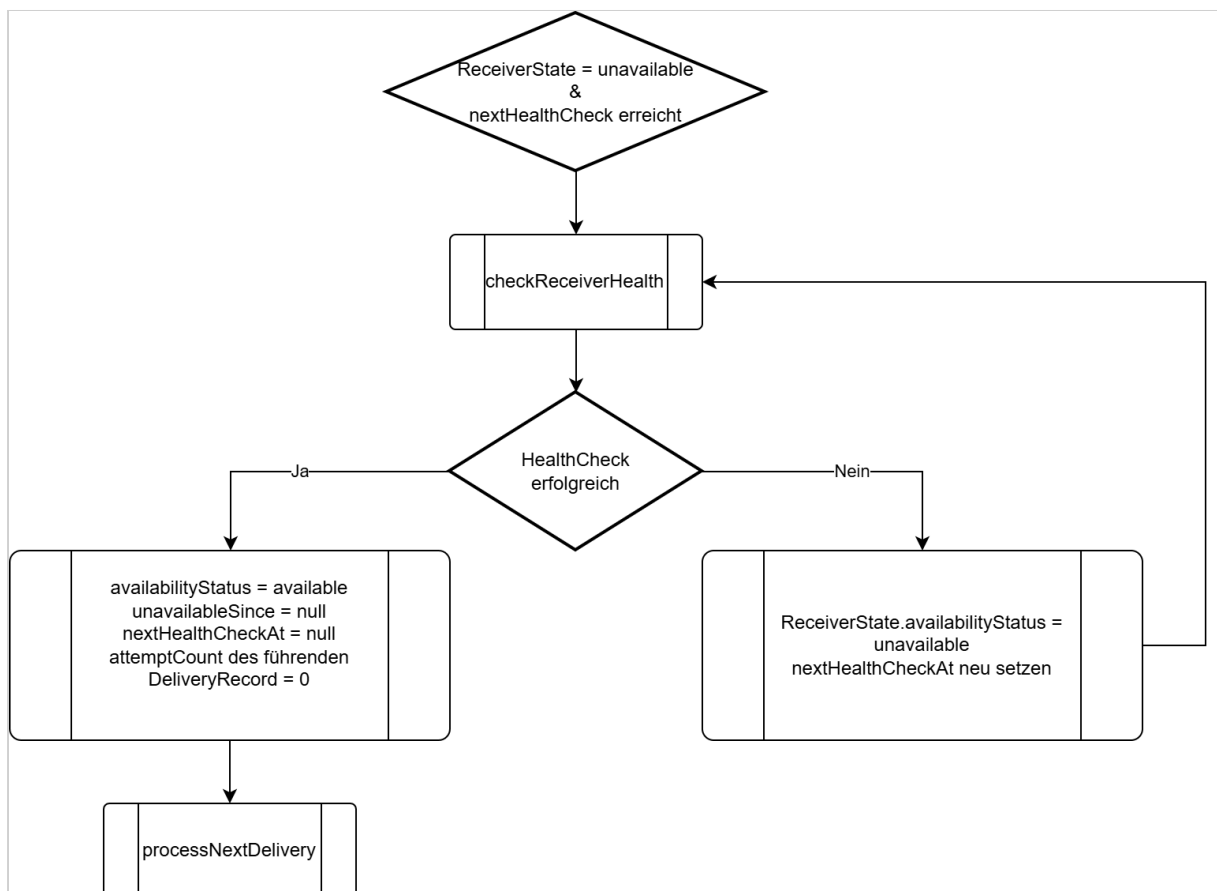
Die erste HTTP-Zustellung wird ohne vorherige Wartezeit ausgeführt. Nach einem temporären Fehler bei den Zustellversuchen 1 - 12 setzt der `DeliveryService` `DeliveryRecord.nextAttemptAt` wie folgt:

- nach den Versuchen 1–3: aktueller Zeitpunkt + 0,5 Sekunden,
- nach den Versuchen 4–6: aktueller Zeitpunkt + 1 Sekunde,
- nach den Versuchen 7–9: aktueller Zeitpunkt + 2 Sekunden,
- nach den Versuchen 10–12: aktueller Zeitpunkt + 10 Sekunden.

Versuch 13 ist der letzte reguläre Zustellversuch eines Zustellzyklus. Schlägt auch dieser Versuch mit einem temporären Fehler fehl, wird der Receiver gemäß der Tabelle "Normative Zustandsübergänge" auf `ReceiverState.availabilityStatus = unavailable` gesetzt.

checkReceiverHealth

Solange `ReceiverState.availabilityStatus = unavailable` ist, werden keine regulären Zustellversuche gestartet. Es werden ausschließlich die geplanten HealthChecks durchgeführt. Das Ergebnis eines HealthChecks wird gemäß der Tabelle "Normative Zustandsübergänge" verarbeitet.

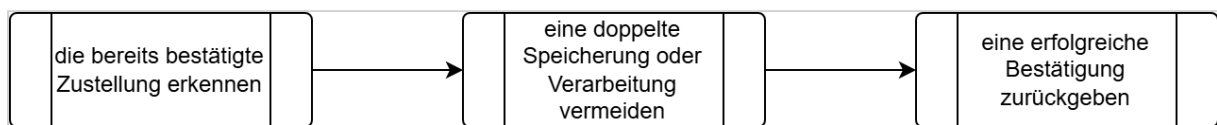


Idempotenz

Die `deliveryId` ist der **Idempotenzschlüssel** des Empfängers, d. h. anhand dieses Schlüssels kann der Empfänger erkennen, ob er exakt diese Zustellung schon einmal empfangen hat oder nicht.

Der Overwatch-Service führt die Duplikaterkennung ausschließlich anhand von `deliveryId` durch. Andere Attribute der erneut zugestellten HTTP-Message werden nicht zur Bestimmung der Identität der Zustellung verwendet. Dies gilt insbesondere für `sentAt`, da dieser Wert bei jedem Zustellversuch unterschiedlich sein kann. Wurde eine `deliveryId` bereits erfolgreich verarbeitet, darf eine erneute Zustellung mit derselben `deliveryId` keinen zweiten Verarbeitungsvorgang auslösen. Der Overwatch-Service muss für das erkannte Duplikat erneut eine erfolgreiche HTTP-Response senden.

Wenn eine doppelte `deliveryId` empfangen wird, muss der Empfänger:



`messageId`

Doppelte HTTP-Zustellungen bleiben möglich, wenn der Empfänger eine Nachricht speichert, die Antwort jedoch verloren geht. Die Idempotenz auf Empfängerseite macht solche Wiederholungen sicher.

Dies ist der Schlüssel, der eine Verbindung zwischen Original- und transformierter Telemetrie herstellt. Sie ist nicht der Idempotenzschlüssel eines Zustellversuchs.

Database-Package

Das Database-Package kapselt den Zugriff auf den persistenten Nachrichtenspeicher des Telemetry Data Adapters. Es enthält keine fachliche Verarbeitungslogik. Die aufrufende Komponente entscheidet, welche Datensätze gelesen oder verändert werden müssen. Das Database Package ist dafür verantwortlich, diese Operationen atomar im persistenten Speicher auszuführen.